

# Notes on Neural Networks: part 1

**Alessio Micheli**

[micheli@di.unipi.it](mailto:micheli@di.unipi.it)

*2024*



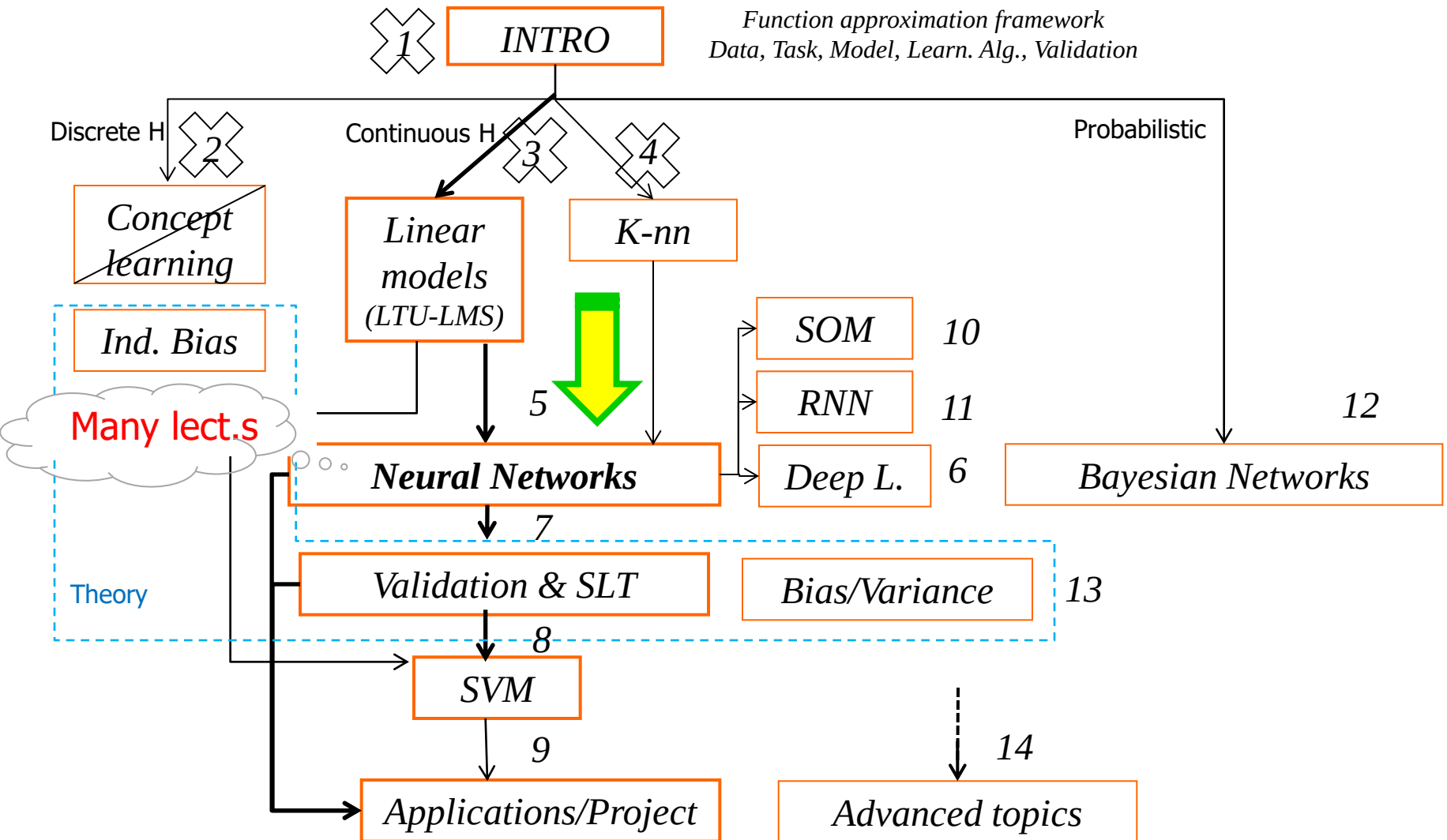
Dipartimento di Informatica  
Università di Pisa - Italy

# ML Course structure

## Where we are



Dip. Informatica  
University of Pisa



# NN: which target?

- NN to study and model biological systems and learning processes
  - Philosophy, Psychology and neurobiology, cognitive science, Computational neuroscience.
  - Biological realism is essential
- NN to introduce effective ML systems/algorithms (often losing a strict biological realism)
  - Statistics, Artificial Intelligence, Physics, Math., Engineering, ...
  - ML, computational and algorithmic properties are essential
- For us: **ANN** (*Artificial* Neural Networks):
  - indeed a (flexible) Machine Learning tool
  - (in the sense of approximation of functions: it realizes a mathematical functions  $h(\mathbf{x})$  with special properties)

# A first look to NN (as ML tool)

---

A historical learning machine,  
while still a powerful approach to ML

- NN can learn from examples
- NN are universal approximators (Theorem of Cybenko):  
flexible approaches for arbitrary functions (including non-linear) !
- NN can deal with noise and incomplete data
  - Performance degrades gracefully under adverse operating conditions
- NN can handle continuous real and discrete data  
for both regression and classification tasks
- Successful model in ML due to the flexibility in applications
- NN encompasses a wide set of models: it is a paradigm !  
(in the class of subsymbolic approaches)

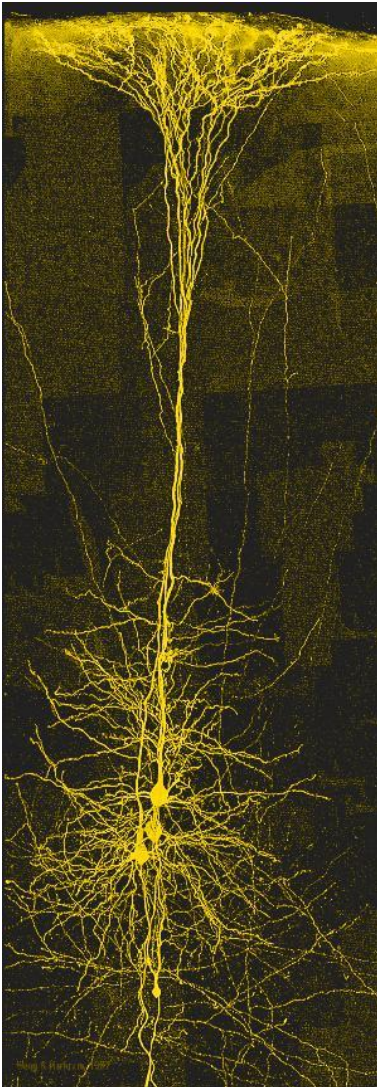
# “Philosophy”: Connectionism

- Complex behavior emerging from the interaction of simple computational units



- Connectionism: mental or behavioural phenomena as the emergent processes of *interconnected networks of simple units*.

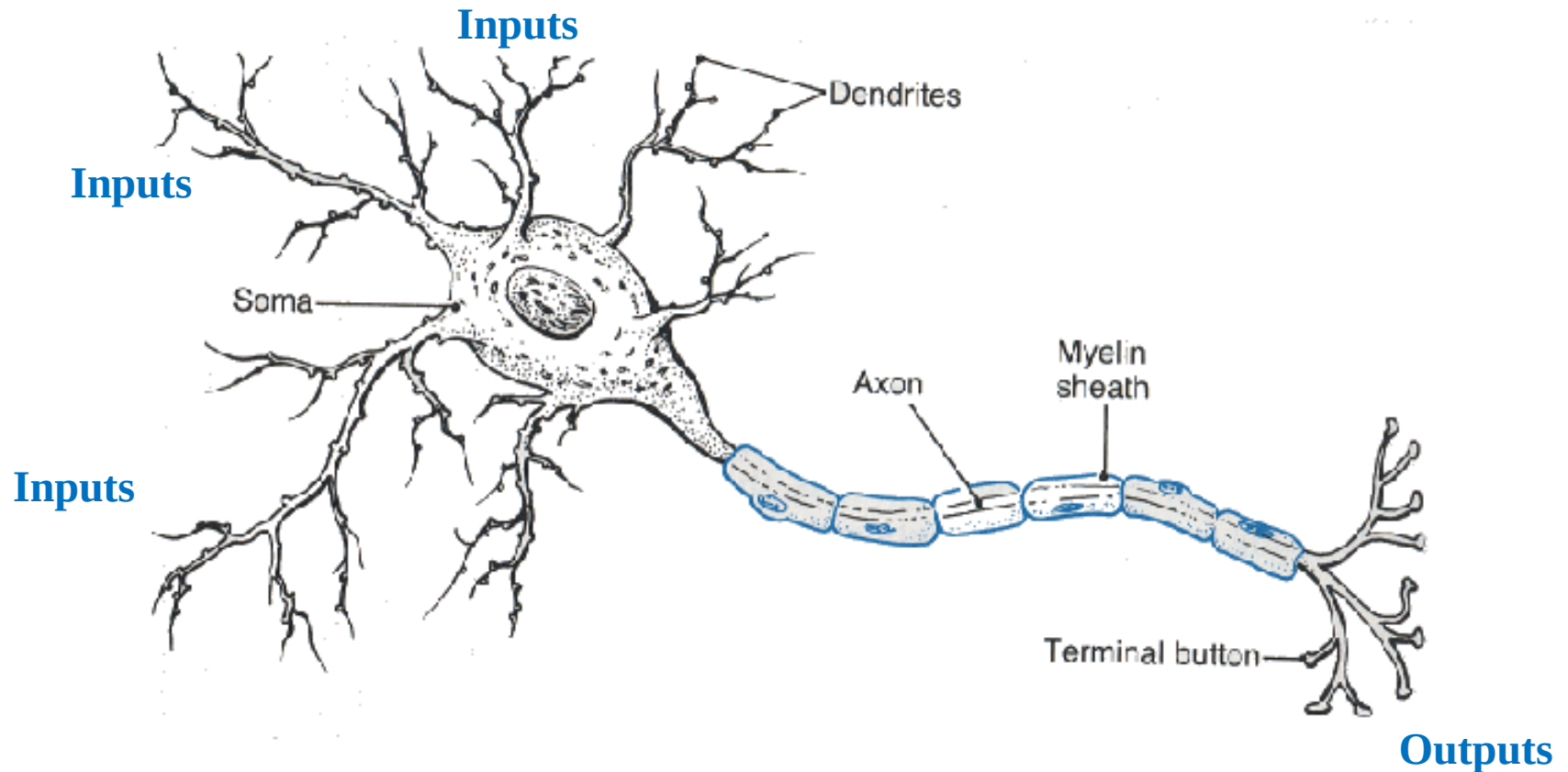
# Nervous system



- Human Brain: more than  $10^{10}$  neurons
- $10^4$ - $10^5$  connections for each neurons
- Response:  $\sim 0.001$  sec. (msec order)
- Image recognition: 0.1 sec (only 100 serial computations)  $\rightarrow$  highly parallel computations
- (Note that face recognition can be complex for conventional computers)

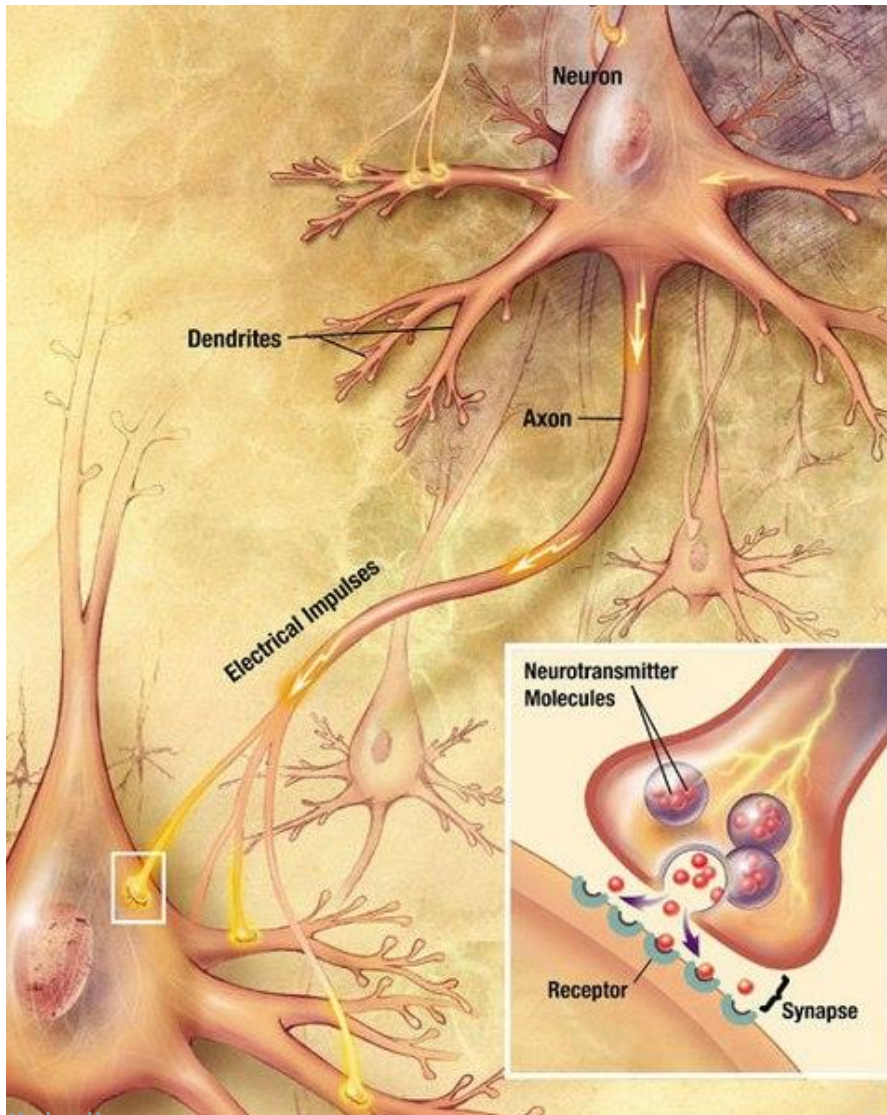
Three stained pyramidal neurons. These neurons stand nearly 2mm high and receive over 10,000 inputs from other neurons which they process in their complex dendritic arbors using active regenerative mechanisms.

# Biological Neuron

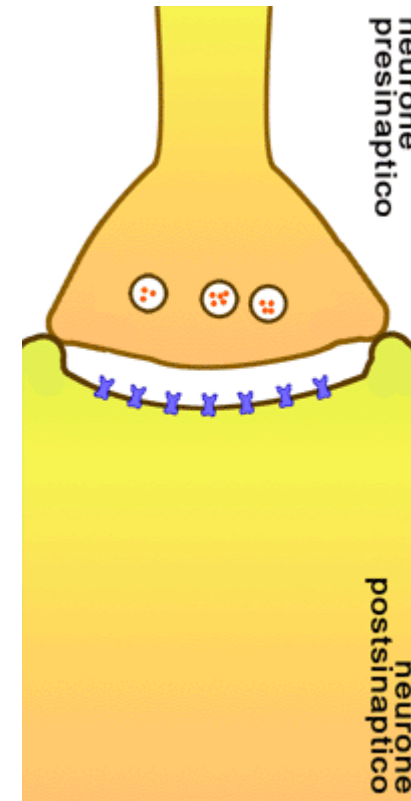


- charge / discharge cycle (sodium / potassium ions)
- changing the potential of the cell (from dendrites), threshold
- spike generation (voltage pulse), time interval among spikes

# Signal propagation and Synapse



A signal propagating down on axon to the cell body and dendrites of the next cell



Electr.

Chem.

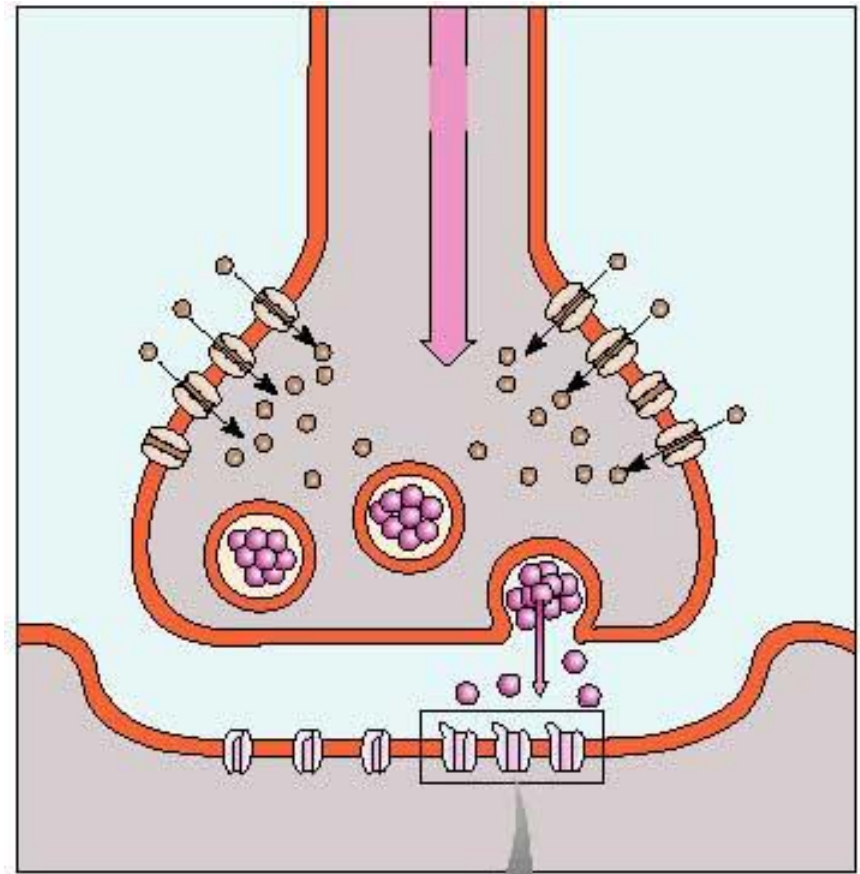
Electr.

# Synapse

- Pre and post synaptic membrane
- Neurotransmitter
- Inhibitory/excitatory

Change due to *learning*:

- Strength (weight) reinforcement due to stimuli
- Plasticity/adaptivity of the nervous system
- Hebbian learning (Hebb 1949)  
A synapse is strengthened when the neuron inputs (and correlated outputs) are repeated. Input stimulates synapsis strength, so *weights become closer to inputs*



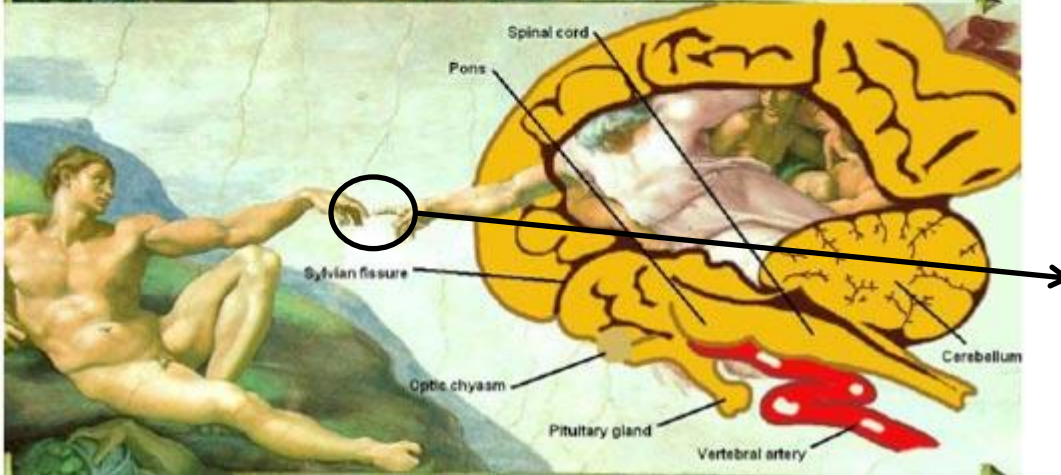
# Neurons in the art

## Michelangelo "The Creation of Adam"

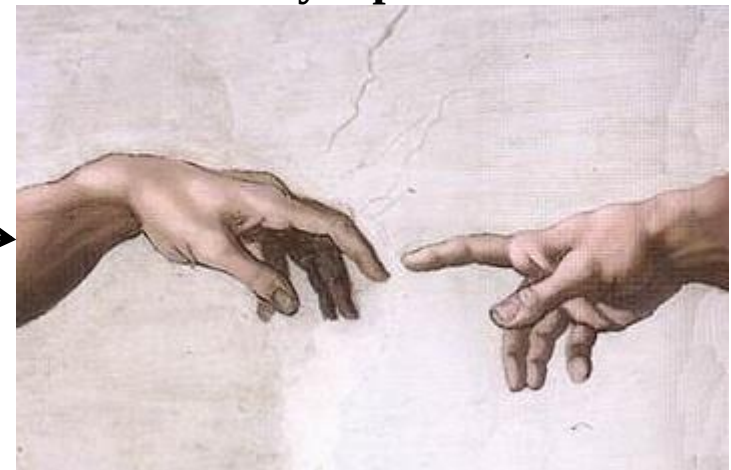


# Neurons in the art

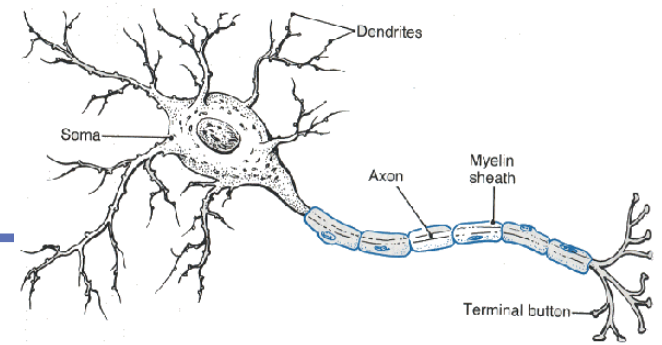
## Michelangelo "The Creation of Adam"



Synapse?

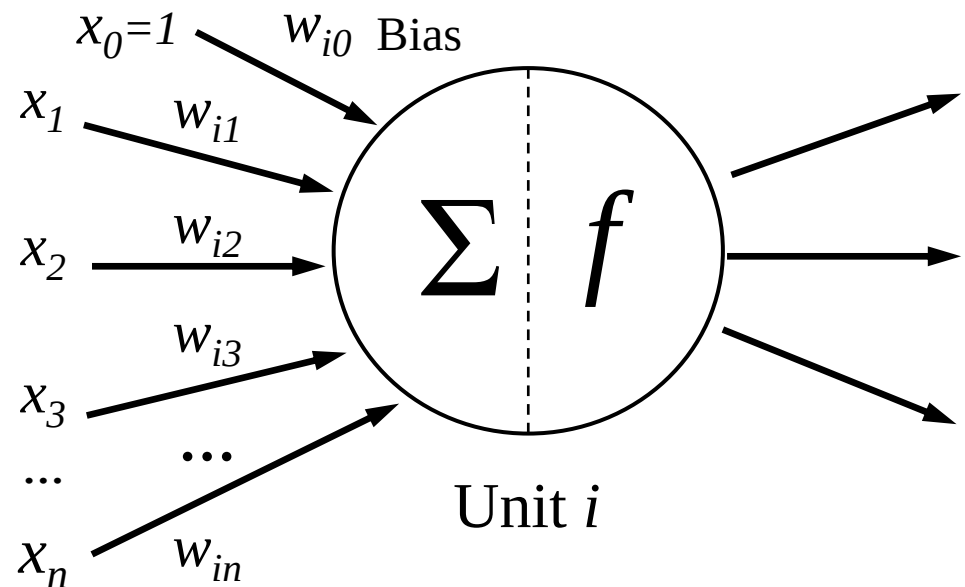


# Artificial Neuron: processing unit



- Node, neuron or **unit**
- Input: from extern source or other units ( $R$ )
- Input Connections: **weights**  $w$ : free parameters: these can be modified by learning (synaptic strength)

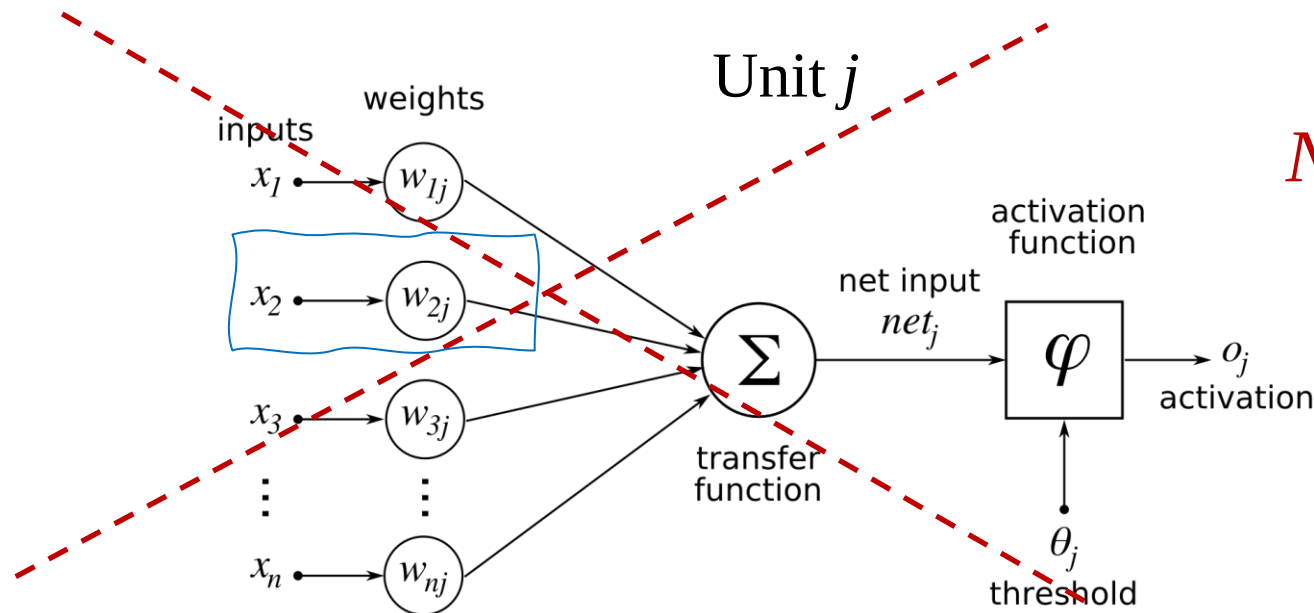
$$\begin{cases} net_i(\mathbf{x}) = \sum_j w_{ij} x_j \\ o_i(\mathbf{x}) = f(net_i(\mathbf{x})) \end{cases}$$



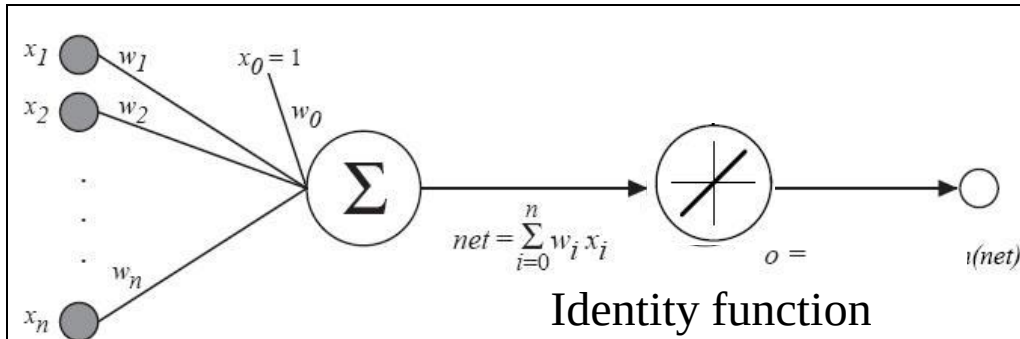
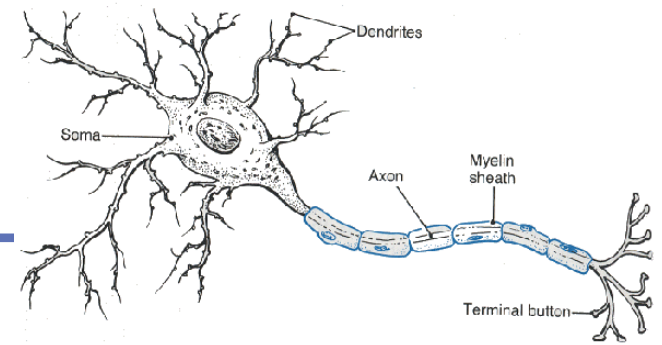
- The weighted sum  $net_i$  is called the **net input** to unit  $i$
- Note that  $w_{ij}$  refers to the weight of the unit  $i$ , i.e. from unit/input  $j$  to unit  $i$  (not the other way around, e.g. see the next slide).
- The function  $f$  is the unit's **activation function (e.g linear, LTU, ...)**.

# Notation for $w_{ij}$

- Some media e.g. Wikipedia (backproagation section) and some NN simulators/libraries use a different notation, whereas, for example  $w_{2j}$  (in the blue box) is the weight from input 2 to unit  $j$ , instead of using as us the traditional  $w_{j2}$  (because  $w_j$  belong to unit  $j$ )
- Take care: We fix the traditional one provided in the previous slide during our course.

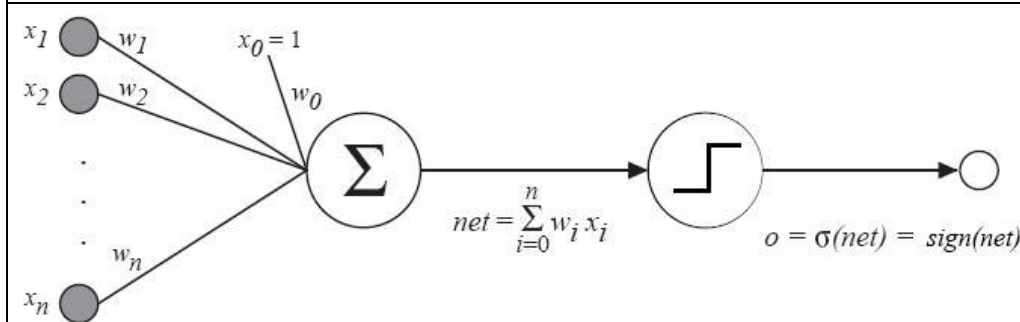


# Neuron: Three activation functions

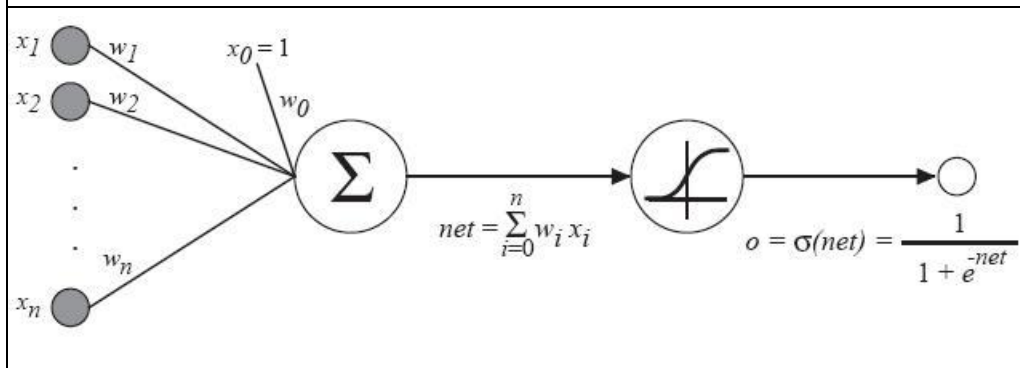


$$h(\mathbf{x}) = \sum_i w_i x_i$$

Linear activation function



**Perceptron** (see LTU)  
Threshold activation function



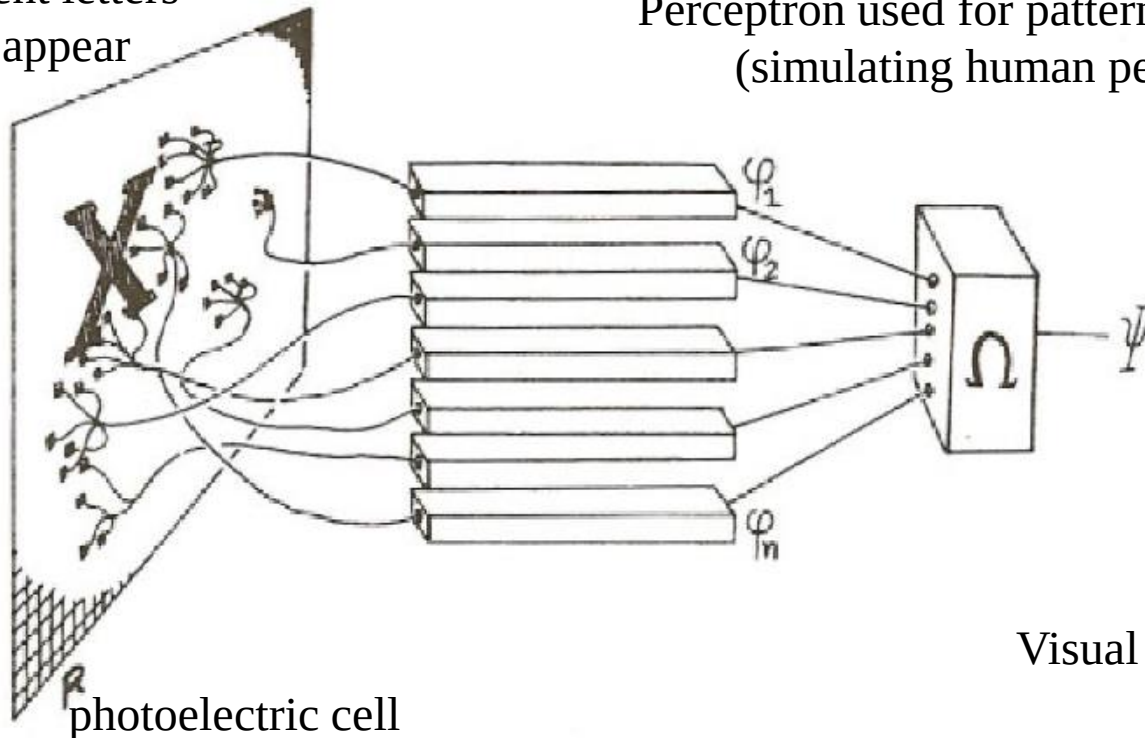
Logistic function : sigmoid  
(see later)

# Perceptron

- Frank Rosenblatt (1957-1958, 1960, ...)

Different letters  
can appear

Perceptron used for pattern classification  
(simulating human perception)



# Perceptron II

- Nice: single neuron is a very simple computational unit
- Minsky: “for the kind of recognition it can do, it is such a simple machine that it would be astonishing if nature did not make use of it somewhere”

Hence, we are going to discuss :

- A biologically inspired model with a simple computational unit (and its learning algorithm)
- that is a model of historical importance for the ML, with different approaches since the early 60s

# Models with Perceptron

- Perceptrons can be composed and connected to build a networks:  
*from LTU to NN !!! (**MLP** NN = Multi Layer Perceptron)*
  - Paradigmatic for the transition from linear to non-linear models towards the flexible models at the state-of-the-art in ML
- *First, by an historical model...McCulloch&Pitts*
- Note: actually NN are realized by software (simulators) or hardware on chips

# McCulloch & Pitts Networks

- “A logical calculus of the ideas immanent in nervous activity” 1943
  - What does a given net compute?
  - Can a given net compute a given logical sentence?
- Neurons are in two possible states: firing (1) and not firing (0)
- All synapses (connections) are equivalent and characterized by a real number (their strength  $w$ ), which is positive for excitatory connections and negative for inhibitory connections;
- a neuron  $i$  becomes active when the sum of those connections  $w_{ij}$  coming from neurons  $j$  connected to it which are active, plus a bias, is larger than zero.
- Binary inputs
- Again binary output (aka binary classification task)
- “Basically” the LTU/Perceptron

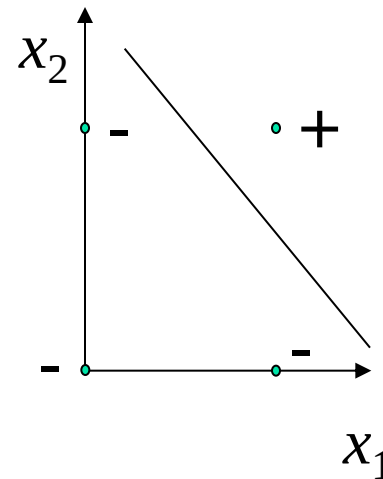
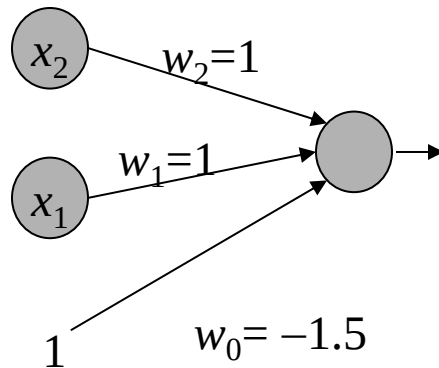
# AND, OR Boolean functions

Assume  $net = \mathbf{w}^T \mathbf{x} = \sum_{i=0}^n w_i x_i$

$$\text{sign}(net) = \begin{cases} 1 & \text{if } net > 0 \\ 0 \text{ (or } -1) & \text{otherwise} \end{cases}$$

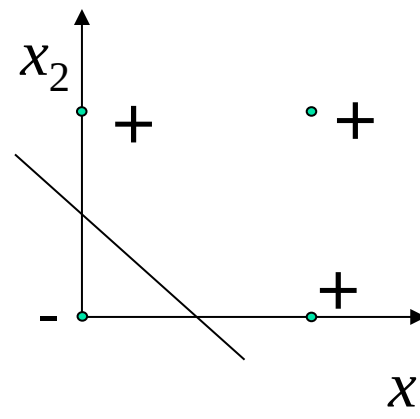
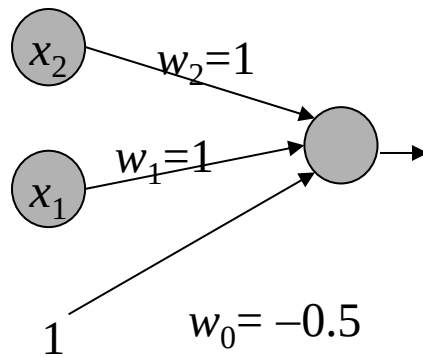
**AND**

| $x_1$ | $x_2$ | $f(x)$ |
|-------|-------|--------|
| 0     | 0     | 0      |
| 0     | 1     | 0      |
| 1     | 0     | 0      |
| 1     | 1     | 1      |



**OR**

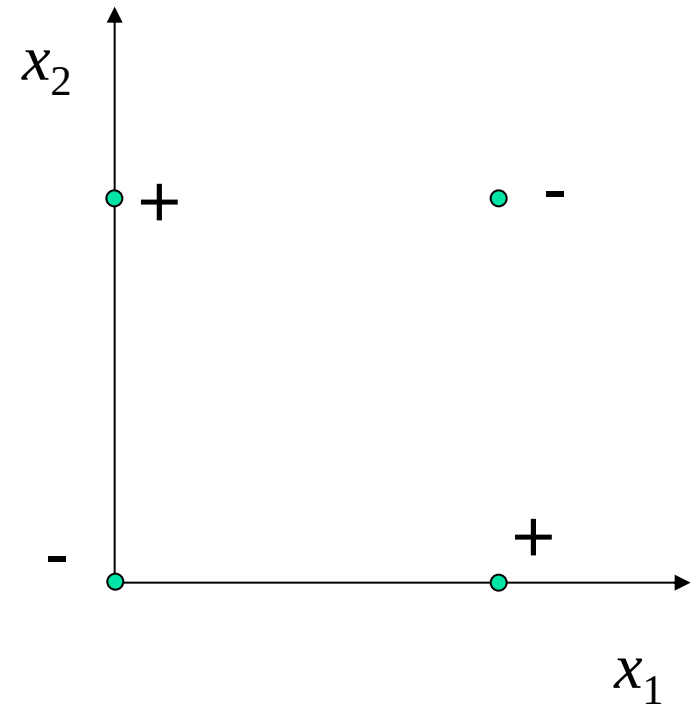
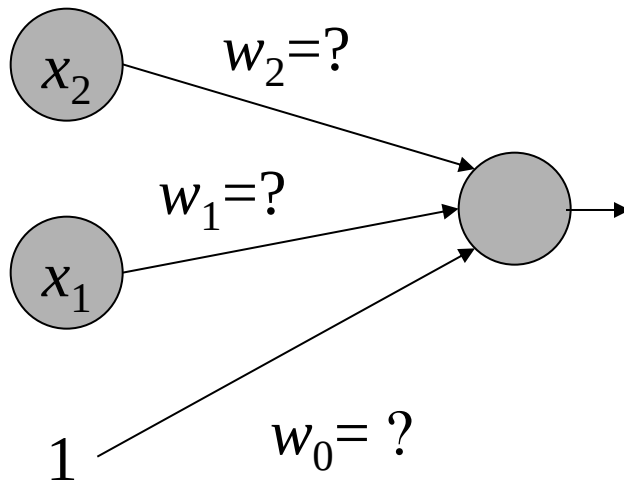
| $x_1$ | $x_2$ | $f(x)$ |
|-------|-------|--------|
| 0     | 0     | 0      |
| 0     | 1     | 1      |
| 1     | 0     | 1      |
| 1     | 1     | 1      |



**Exercise:** find a network for the NOT

# Exclusive OR (XOR)

| $x_1$ | $x_2$ | $f(x)$ |
|-------|-------|--------|
| 0     | 0     | 0      |
| 0     | 1     | 1      |
| 1     | 0     | 1      |
| 1     | 1     | 0      |

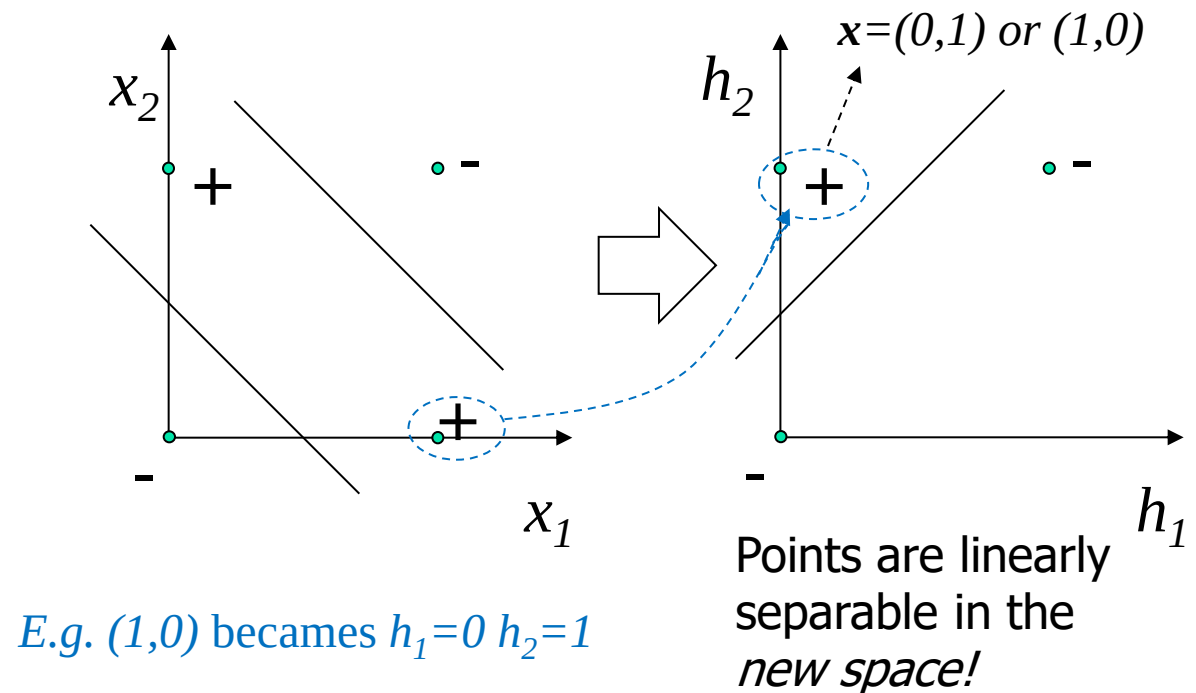
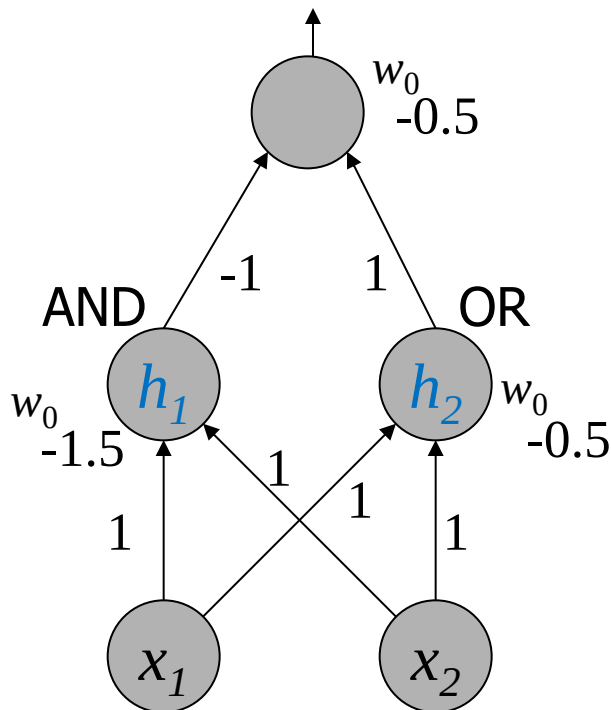


No linear separation  
surface exists!!  
Minsky and Papert (1969)

# XOR by a Two Layers Network

$$x_1 \oplus x_2 = x_1 \cdot \overline{x_2} + \overline{x_1} \cdot x_2 \quad \text{let : } \begin{aligned} h_1 &= x_1 \cdot x_2 & (\text{dot} = \mathbf{and}) \\ h_2 &= x_1 + x_2 & (+ = \mathbf{or}) \end{aligned}$$

then we have :  $x_1 \oplus x_2 = \overline{h_1} \cdot h_2$  (next slide)



**Exercise:** separate by other planes → find different networks solving XOR 21

# Details to solve the XOR (proof)



Dip. Informatica  
University of Pisa

- DE MORGAN rule :  $\text{not}(a \text{ and } b) = \text{not}(a) \text{ or } \text{not}(b)$
- Hence, if  $h_1 = (a \text{ and } b) \rightarrow \text{not}(h_1) = (\text{not}(a) \text{ or } \text{not}(b))$
- And if  $h_2 = (a \text{ or } b) \rightarrow$   
 $\text{not}(h_1) \text{ and } h_2 = (\text{not}(a) \text{ or } \text{not}(b)) \text{ and } (a \text{ or } b) =$   
 $= (\text{not}(a) \text{ and } a) \text{ or } (\text{not}(a) \text{ and } b) \text{ or } (a \text{ and } \text{not}(b)) \text{ or}$   
 $(b \text{ and } \text{not}(b)) =$   
 $= (\text{not}(a) \text{ and } b) \text{ or } (a \text{ and } \text{not}(b)) = (\text{by xor def.})$   
 $= a \text{ xor } b \text{ (C.V.D.)}$

Exercise: redo by yourself with  $x$ ,  $+$ , etc



# Hidden layer & Representation

- Useful concept of internal **(re)representation** of input variables via internal (hidden) units
- Developing *high level (hidden) features* is a key factor in NN
- The representation in the hidden layer makes easier the task to the output layer (last unit).
  - E.g. See the figure on the right: point  $(1,0)$  (in the sud-est corner) becamas  $h_1=0$   $h_2=1$  (north-west corner)  $\rightarrow$  now it is a linear separable problem

A look ahead:

- Such **composition** of sub-/intermediate operations to perform a complex task can be extended through *many layers* of abstraction
- In NN such internal representation/intermediate features can be **learned**
- Learning internal distributed representation  $\rightarrow$  *representation learning* and *deep learning* concepts (link to next parts of the ML course)

# McCulloch&Pitts Summary

- Properties of Networks of Perceptrons:
- Perceptrons can represent AND, OR, NOT (NAND, NOR)
- → every Boolean function can be represented by some network of interconnected perceptrons.
- Only **two** levels deep is sufficient (or more can be more efficient\*).
- ... but **single** layer cannot model all the possible functions (Minsky&Papert 1969): limitations of single perceptrons due to the linear separable problems!
- Note: **no** provided (up to now) learning algorithm (even for one unit)!!!!
- Let us start (again) with a single unit model

# Learning for one unit model - Overview



Dip. Informatica  
University of Pisa

Two kinds of method for learning (also historical view):

1. **Adaline** = Adaptive Linear Neuron (Widrow, Hoff):  
linear unit during training: LMS direct solution and gradient descent solution
  - Regression tasks: See the LMS algorithm
  - For classification: See the LTU and LMS algorithm of a previous lectures (on linear model)
  - An approach that we will generalize to MLP
2. **Perceptron** (Rosenblatt): non-linear unit during training: with hard limiter or Threshold activation function
  - Only classification: Capabilities studies, convergence theorem
  - Next slides

# Introduction to 2.: the Perceptron Learning Alg.



Dip. Informatica  
University of Pisa

- Rosenblatt's Perceptron Learning Algorithm (1958-1962)
- Minimize the number of misclassified patterns:
  - find  $w$  s.t.  $\text{sign}(w^T x) = d$
  - On-line algorithm: a step can be made for each input pattern
  - Note that to build linear classifier we will have from now **3 learning alg:**
    1. Adaline LMS direct solution
    2. Adaline LMS gradient based alg.
    3. Perceptron learning algorithm (now)
      - And others will come (later in the course, e.g. SVM)



# The “Perceptron Learning Algorithm”



Dip. Informatica  
University of Pisa

1. Initialize the weights (either to zero or to a small random value)
2. pick a learning rate  $\eta$  (this is a number between 0 and 1)
3. until stopping condition is satisfied (e.g. weights don't change):

For each training pattern  $(\mathbf{x}, d)$  ( $d = +1$  or  $-1$ ) [let  $out$  be the output]:

- Compute output activation  $out = \text{sign}(\mathbf{w}^T \mathbf{x})$   $[+1, -1]$
- If  $out = d$ , don't change weights I.e. “minimize (only) misclassifications”
- If  $out \neq d$ , update the weights:
  - $\mathbf{w}_{\text{new}} = \mathbf{w} + \eta d \mathbf{x}$ 
    - add  $+\eta \mathbf{x}$  if  $\mathbf{w}^T \mathbf{x} \leq 0$  and  $d = +1$
    - $\eta \mathbf{x}$  if  $\mathbf{w}^T \mathbf{x} > 0$  and  $d = -1$

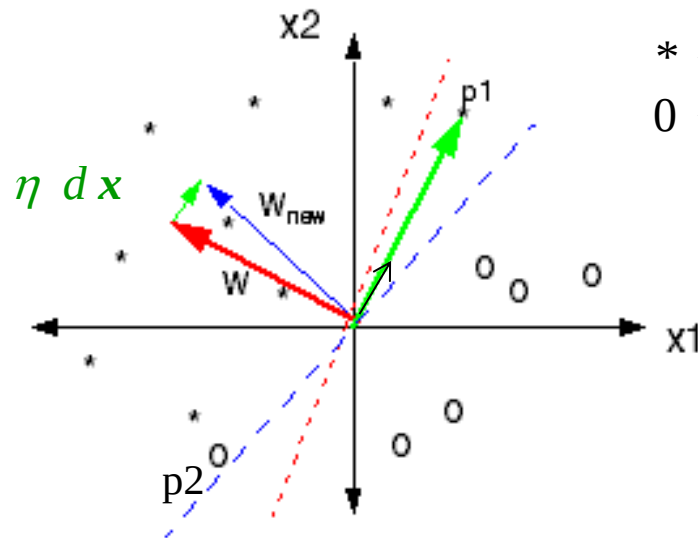
Or (in a different form):

- $\mathbf{w}_{\text{new}} = \mathbf{w} + 1/2 \eta (d - out) \mathbf{x}$

Note:  $\Delta \mathbf{w}$  in LMS was different because we have here the  $\text{sign}()$  in the  $out$  computation

# Geometrical view

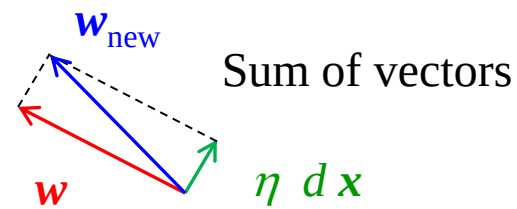
**An example:** Before updating the weight  $w$  (pointing the positive region), we note that both  $p1$  and  $p2$  are incorrectly classified (the red dashed line is decision boundary). Suppose we choose  $p1$  to update the weights as in picture below.  $p1$  has target value  $d=1$ , so that  $w$  is moved a small amount in the direction of  $p1$ . The new boundary (blue dashed line) is better than before (and  $w_{\text{new}}$  closer to  $p1$ ). **Exercise:** what happens if the training pattern  $p2$  is chosen ?



$* \rightarrow d=+1$   
 $0 \rightarrow d=-1$

$$w_{\text{new}} = w + \eta d x$$

Note: only for this example ( $w_0$ ) bias = 0



Note the *Hebbian rule*:  $w$  moves towards  $x$

# Delta Rule

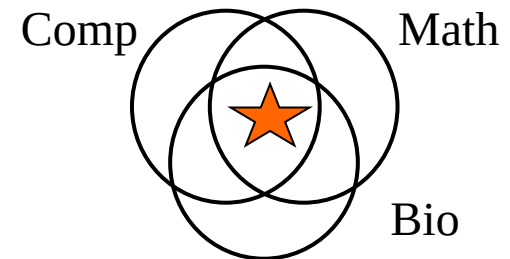
- The form  $\mathbf{w}_{\text{new}} = \mathbf{w} + \eta d \mathbf{x}$  is seen as *Hebbian learning*
- The other form  $\mathbf{w}_{\text{new}} = \mathbf{w} + \eta (d - \text{out}) \mathbf{x} = \mathbf{w} + \eta \delta \mathbf{x}$   
is in the form of *error-correction* learning
- Recall from LMS: This is an “*error correction*” rule (**delta/** Widrow-Hoff/**Adaline/LMS rule**) that changes the  $\mathbf{w}$  proportionally to the error (**target  $d$ -output**):
  - E.g. (target  $d$  – output) = err=0  $\rightarrow$  no correction
  - (input>0) if err + (output is too low), increase  $\mathbf{w} \rightarrow$  incr.  $\mathbf{w}^T \mathbf{x} \rightarrow$  reduce err
  - ...
- In terms of “neurons”: *the adjustment made to a synaptic weight is proportional to the product of error signal and the input signal that excite the synapse*
- Easy to compute when errors signal  $\delta$  is directly measurable (we know the desired response for each unit)

# Represent / Learn

- What can Perceptron represent? linear decision boundaries (see LTU)
  - Perceptron can solve linearly separable problems
- Can it always learn the solution?
- Yes! The Perceptron with the “Perceptron Learning Algorithm” is always able to learn what it is able to represent:

## *Perceptron Convergence Theorem*

*Proof in the next lesson!*



*Note: A milestone results!!!! A biologically (Bio) inspired model with well-defined and proved computational capabilities (Comp)!!! And proved by a theorem (Math)*

# Perceptron Convergence Theorem



Dip. Informatica  
University of Pisa

- *The perceptron is guaranteed to converge (classifying correctly all the input patterns) in a finite number of steps if the problem is linearly separable.*
- (independently of the starting point, although the final solution is not unique and it depends on the starting point)
- May be “unstable” if the problem is not separable
  - In particular it develop cycles (with a set of weights that are not necessarily optimal)

Note: I'll simplify the notations in the proof (e.g. assume the *not* reported scalar product with  $\top$  as usual among vectors)

# Perceptron Convergence

## Theorem: preliminaries (1)



Dip. Informatica  
University of Pisa

We can focus on a “all positive patterns” task (as shown in the following):

Assume  $(\mathbf{x}_i, d_i)$  in the TR set, with  $d_i = +1$  or  $-1$  and  $i=1\dots l$

Linearly separable  $\rightarrow \exists \mathbf{w}^*$  solution s.t. (omitting  $T$ )

$$d_i (\mathbf{w}^* \mathbf{x}_i) \geq \alpha, \text{ with } \alpha = \min_i d_i (\mathbf{w}^* \mathbf{x}_i) > 0$$

Hence,  $\mathbf{w}^* (d_i \mathbf{x}_i) \geq \alpha$

Defining  $\mathbf{x}_i' = (d_i \mathbf{x}_i)$ , then  $\mathbf{w}^*$  is a solution iff

$$\mathbf{w}^* \text{ is a solution of } (\mathbf{x}_i', +1)$$

(if)  $\mathbf{w}^*$  solve  $\rightarrow d_i (\mathbf{w}^* \mathbf{x}_i) \geq \alpha \rightarrow (\mathbf{w}^* d_i \mathbf{x}_i) \geq \alpha \rightarrow (\mathbf{w}^* \mathbf{x}_i') \geq \alpha \rightarrow \mathbf{w}^*$  is a solution of  $(\mathbf{x}_i', +1)$

(only if)  $\mathbf{w}^*$  is a solution of  $(\mathbf{x}_i', +1) \rightarrow (\mathbf{w}^* d_i \mathbf{x}_i) \geq \alpha \rightarrow d_i (\mathbf{w}^* \mathbf{x}_i) \geq \alpha \rightarrow \mathbf{w}^*$  solve for  $\mathbf{x}_i$

# Perceptron Convergence

## Theorem: preliminaries (2)



Dip. Informatica  
University of Pisa

Assuming  $w(0)=0$  (at step 0),

$$\eta = 1,$$

$$\beta = \max_i \|\mathbf{x}_i\|^2 \quad (i=1\dots l, \text{ and } \|\cdot\| \text{ denotes the Euclidian norm, see first lesson})$$

After  $q$  errors (all false negative)

$$\mathbf{w}(q) = \sum_{j=1 \rightarrow q} \mathbf{x}_{(i_j)}$$

$i_j$  is used to denote the patterns belonging to the subset of misclassified patterns e.g., the indices 2, 8, 9, 14 in the TR set.

$$\text{Because } \mathbf{w}(j) = \mathbf{w}(j-1) + \mathbf{x}_{i_j}$$

**Rule:**  $\mathbf{w}_{\text{new}} = \mathbf{w} + \eta \, d \, \mathbf{x}$  (but all  $d$  are +1 here, only positive patterns, and  $\eta=1$ )

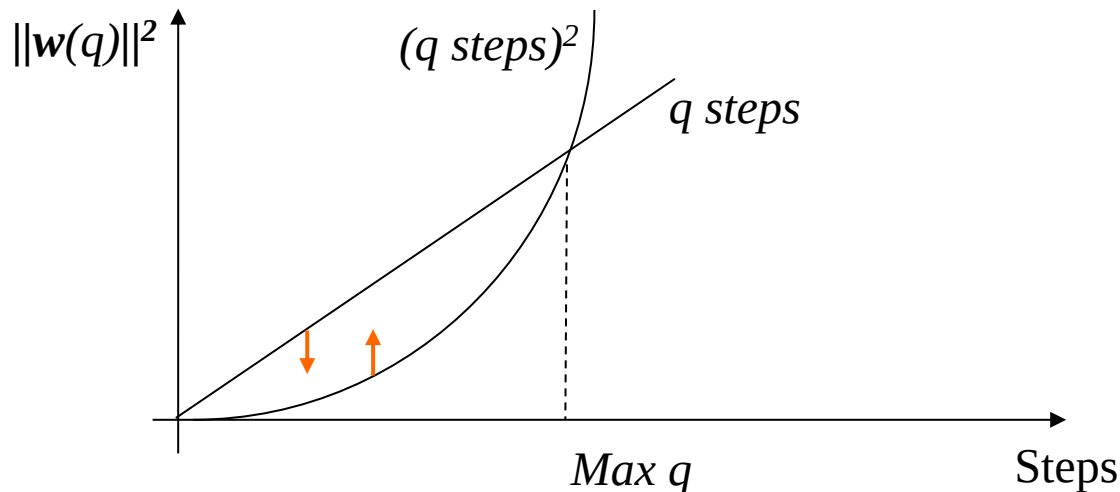
# Perceptron Convergence Theorem: Proof



Dip. Informatica  
University of Pisa

**Theorem** “For linearly separable tasks, the perceptron algorithm converges after a finite number of steps”

**Basic idea:** we can find lower and upper bound to  $\|w(q)\|^2$  as a function of  $q^2$  steps (lower bound  $\uparrow$ ) and  $q$  steps (upper bound  $\downarrow$ )  $\rightarrow$  we can find  $q$  (number of steps) s.t. the algorithm converges:  
[Proof also in Haykin 2<sup>nd</sup> ed. pages 139-141]



# Perceptron Convergence

## Theorem: Proof (part 1)



Dip. Informatica  
University of Pisa

- Lower bound on  $\|\mathbf{w}(q)\|^2$ :

$$(\mathbf{w}^*)^T \mathbf{w}(q) = (\mathbf{w}^*)^T \sum_{j=1 \rightarrow q} \mathbf{x}_{i_j} \geq q\alpha$$

Recall that :

- $\mathbf{w}^*$  is the solution
- $\alpha = \min_i ((\mathbf{w}^*)^T \mathbf{x}_i)$

Cauchy-Swartz:  $(\mathbf{a}^T \mathbf{b})^2 \leq \|\mathbf{a}\|^2 \|\mathbf{b}\|^2 \implies \|\mathbf{a}\|^2 \|\mathbf{b}\|^2 \geq (\mathbf{a}^T \mathbf{b})^2$

Use  $\mathbf{a} = \mathbf{w}^*$  and  $\mathbf{b} = \mathbf{w}(q)$

$$\|\mathbf{w}^*\|^2 \|\mathbf{w}(q)\|^2 \geq ((\mathbf{w}^*)^T \mathbf{w}(q))^2 \geq (q\alpha)^2$$

$$\underbrace{\hspace{1.5cm}} \longrightarrow \|\mathbf{w}(q)\|^2 \geq (q\alpha)^2 / \|\mathbf{w}^*\|^2$$

The lower bound

# Proof (part 2)

$$\|\mathbf{a} + \mathbf{b}\|^2 = \sum_i (a_i + b_i)^2 = \sum_i a_i^2 + \sum_i 2a_i b_i + \sum_i b_i^2 = \|\mathbf{a}\|^2 + 2\mathbf{a}\mathbf{b} + \|\mathbf{b}\|^2$$

- Upper bound on  $\|\mathbf{w}(q)\|^2$ :

$$\|\mathbf{w}(q)\|^2 = \|\mathbf{w}(q-1) + \mathbf{x}_{i_q}\|^2 = \|\mathbf{w}(q-1)\|^2 + 2\mathbf{w}(q-1)\mathbf{x}_{i_q} + \|\mathbf{x}_{i_q}\|^2$$

For the  $q$ -th error: *this is*  $< 0$ , hence, deleting a negative term:

$$\|\mathbf{w}(q)\|^2 \leq \|\mathbf{w}(q-1)\|^2 + \|\mathbf{x}_{i_q}\|^2$$

And by iteration ( $\mathbf{w}(0)=0$ )

$$\|\mathbf{w}(q)\|^2 \leq \sum_{j=1 \rightarrow q} \|\mathbf{x}_{i_j}\|^2 \leq q\beta$$

$$\beta = \max_i \|\mathbf{x}_i\|^2$$

The upper bound

# Proof (part 3): bring them together



Dip. Informatica  
University of Pisa

Upper bound

Lower bound

$$q\beta \geq ||\mathbf{w}(q)||^2 \geq (q\alpha)^2 / ||\mathbf{w}^*||^2$$

$$q\beta \geq q^2 \alpha'$$

$$\beta \geq q \alpha'$$

$$\boxed{q \leq \beta / \alpha'}$$

*i.e.* a finite number of steps

# Differences between “Perc. Learning Alg.” and LMS Alg. (I)



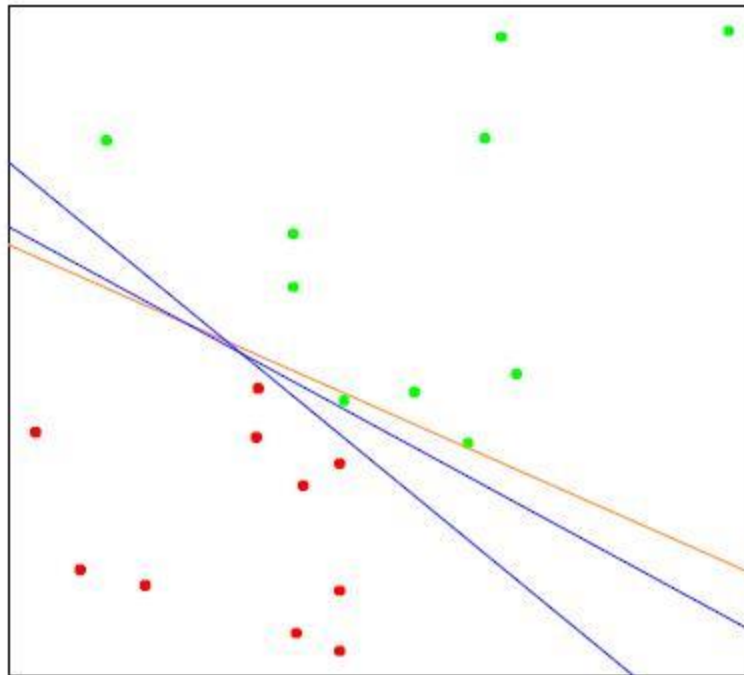
Dip. Informatica  
University of Pisa

$$\mathbf{w}_{\text{new}} = \mathbf{w} + \eta (d - \text{out}) \mathbf{x}$$

- Apparently similar but note that:
  - LMS rule was derived (by the gradient) without threshold activation functions: minimization of the error of the linear unit (using directly  $\mathbf{w}^T \mathbf{x}$ )
  - The perceptron use  $\text{out} = \text{sign}(\mathbf{w}^T \mathbf{x})$
- Hence, for training:
  - $\delta = (d - \mathbf{w}^T \mathbf{x})$  for the LMS approach  
versus
  - $\delta = (d - \text{sign}(\mathbf{w}^T \mathbf{x}))$  for the perceptron learning alg.
- Of course the model trained with LMS can still be used for classification applying the threshold function  $h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$  (LTU)

# LMS can misclassify in linear separable problems

- LMS not necessarily minimize the number of TR examples misclassified by the LTU



$$E(\mathbf{w}) = \sum_p (y_p - \mathbf{x}_p^T \mathbf{w})^2$$

It changes the weights **not only** for the misclassified patterns!

**FIGURE 4.14.** A toy example with two classes separable by a hyperplane. The orange line is the least squares solution, which misclassifies one of the training points. Also shown are two blue separating hyperplanes found by the perceptron learning algorithm with different random starts.

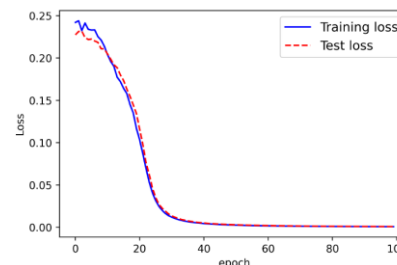
# Differences Summary

## Perceptron Learn. Alg.

- Min. misclassifications ( $out = \text{sign}(\mathbf{w}^T \mathbf{x})$ )
- it always converges for linear separable problems (in a finite number of steps) to a perfect classifier
- (else no convergence)
- difficult to be extended to networks of units (NN)

## LMS Alg.

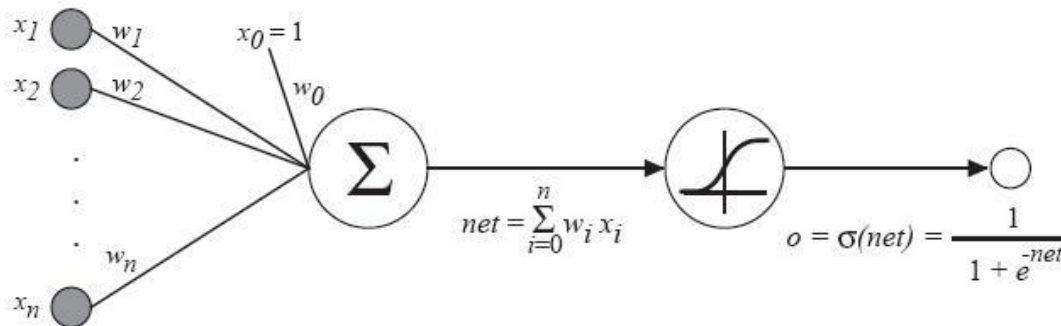
- Min.  $E(\mathbf{w})$  with  $out = \mathbf{w}^T \mathbf{x}$
- asymptotic convergence (\*), also for not linear separable problems
- not always zero classification errors, linear separable problems
- it can be extended to networks of units (NN), using the gradient based approach



(\*) example of *asymptotic* convergence  
(training with MSE loss)

# Activation functions

1. Linear function
2. Threshold (or step) function (Perceptron/LTU)
3. A non-linear *squashing* function like the **sigmoidal** logistic function: assumes a continuous range of values in the bounded interval  $[0,1]$



$$f_{\sigma}(x) = \frac{1}{1 + e^{(-ax)}}$$

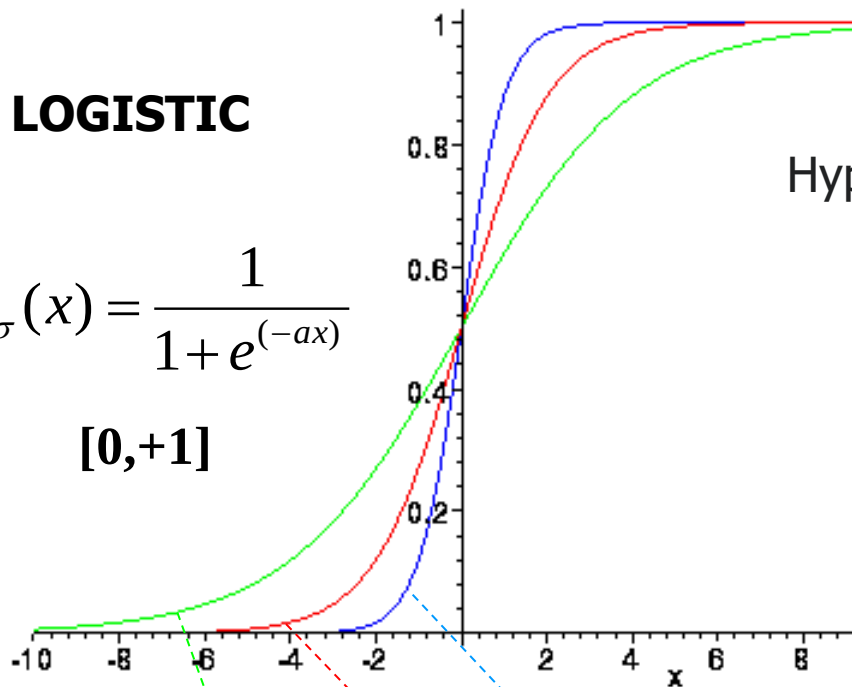
- The sigmoidal-logistic function has the property to be a *smoothed differentiable threshold function*.
- $a$  is the *slope* parameter of the sigmoid function

# Activation functions (type 3.)

## LOGISTIC

$$f_{\sigma}(x) = \frac{1}{1 + e^{(-ax)}}$$

$[0, +1]$



$a=0.5$  green,  $a=1$  red,  $a=2$  blue

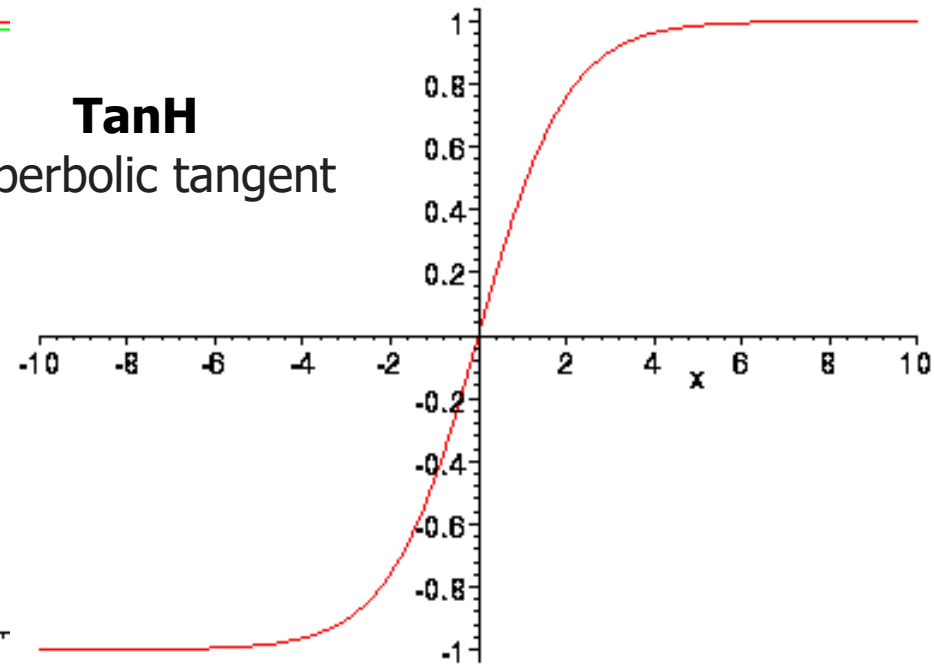
Semilinearity close to 0

What if  $a \rightarrow 0$ ? (to linear)

$a \rightarrow \infty$ ? (to LTU)

## TanH

Hyperbolic tangent



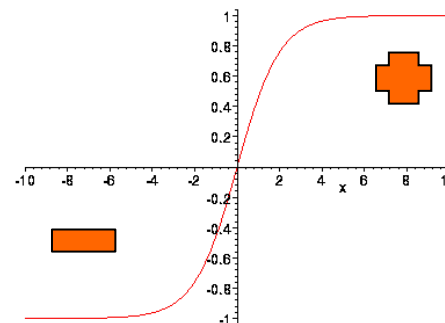
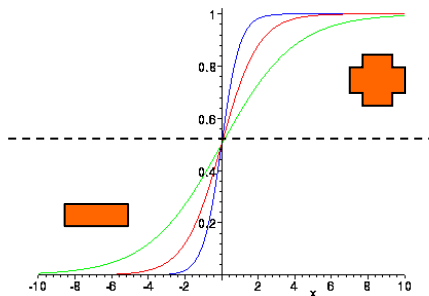
$$f_{\text{symm}} = 2 f_{\sigma}(x) - 1 = \tanh(ax/2)$$

$[-1, +1]$

# Sigmoidal functions and classifier outcomes

These functions provide continuous outputs but...

- For the Logistic function an output value
  - $\geq 0.5$  (**threshold**) correspond to the **positive class**
  - $< 0.5$  correspond to the **zero or negative class**
- It is possible to change this threshold (e.g. studying the effect on FP/FN etc, or by a ROC),
- and even to consider a *rejection zone* in an interval around the threshold value (to avoid fragile decisions)
- For the TanH the **threshold is in 0** (with the analogous possibilities)

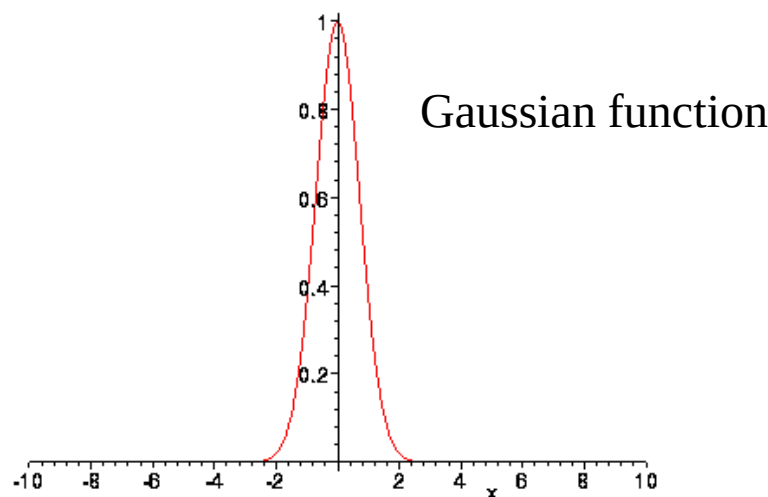


# Other Activation Functions

- Radial Basis Functions  $\rightarrow$  *RBF networks*

$$f(x) = e^{(-ax^2)}$$

$$\|w - x_{input}\|$$



- Softmax: see next lectures (multi-output case)
- Stochastic neurons: output is +1 with probability  $P(net)$  or -1 with  $1 - P(net) \rightarrow$  *Boltzmann machines and other models rooted in statistical mechanics*



# Other Activation Functions (2)

- Tanh-like (piecewise linear approximation) for efficient computation

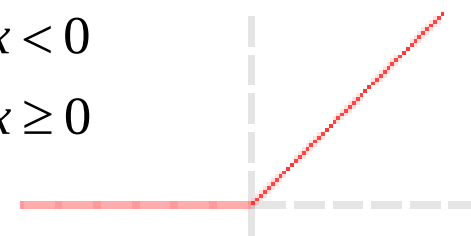
$$f(x) = \text{sign}(x) \left[ 1 + \frac{1}{2^{2^n |x|}} \left( \frac{2^{2^n |x|} - \lfloor 2^{2^n |x|} \rfloor}{2} - 1 \right) \right]$$

$n$  steepness (slope) of the function  
Rounding operator (floor)  
Shift operation in binary rep.

- Rectifier → **ReLU** (Rectified Linear Unit): see *Deep Learning* section

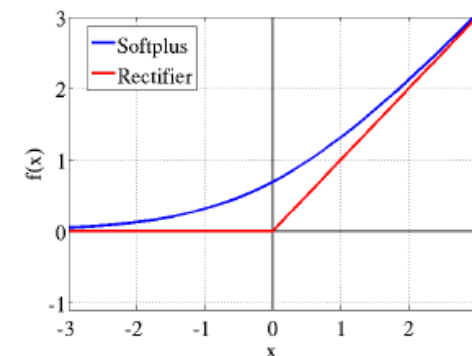
It has become a default choice for Deep models, so it (and its variants) deserves more attention discussing Deep Learning

$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$
$$f(x) = \max(0, x)$$



- Softplus** (smooth approximation)

$$f(x) = \ln(1 + e^x)$$



- An updated list (Note: many perform comparably!):

[https://en.wikipedia.org/wiki/Activation\\_function](https://en.wikipedia.org/wiki/Activation_function)

# Activation functions: derivatives



Dip. Informatica  
University of Pisa

- The derivative of the **identity function** is 1.
- The derivative of the **step** (threshold) **function** is not defined, which is exactly why it isn't used with LMS .
- **Sigmoids**: for asymmetric and symmetric case we have ( $a=1$ ):

$$\frac{df_{\sigma}(x)}{dx} = f_{\sigma}(x)(1 - f_{\sigma}(x))$$

$$\frac{df_{\tanh}(x)}{dx} = 1 - f_{\tanh}(x)^2$$

**Show by exercise:** also with any  $a$

# LMS with $f_\sigma$

- The sigmoidal-logistic function has the property to be a *smoothed differentiable threshold function*
- Hence, we can derive a Least (Mean) Square algorithm by computing the gradient of the mean square loss function as for the linear units (also for a classifier)
- From  $o(\mathbf{x}) = \mathbf{x}^T \mathbf{w}$  to  $o(\mathbf{x}) = f_\sigma(\mathbf{x}^T \mathbf{w})$  where  $f_\sigma$  is a logistic function
- Find  $\mathbf{w}$  to *minimize* the residual sum of squares:

$$E(\mathbf{w}) = \sum_p (d_p - o(\mathbf{x}_p))^2 = \sum_p (d_p - f_\sigma(\mathbf{x}_p^T \mathbf{w}))^2$$

# LMS with 1 non-linear unit

## Compute the gradient



Dip. Informatica  
University of Pisa

**Important Exercise:** compute the gradient of the error function

(do it before the  
next lecture)

$$\frac{\partial E(\mathbf{w})}{\partial w_j}$$

with  $o(\mathbf{x}_p) = out(\mathbf{x}_p) = f_\sigma(\mathbf{x}_p^T \mathbf{w}) = f_\sigma(net(\mathbf{x}_p))$

- directly (by composition of functions)

- or by the chain rule  $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial x}$

Whereas  $E$  is  $f$  and  $w$  is  $x$ ,  
using *first*: auxiliary  $g = net$ ,  
and then auxiliary  $g = out$

- **Suggestion**: compute it starting with the singular  $E_p$  and sum the derivatives to obtain the total for  $l$  data

- Interpretation: This derivatives answers to: *How much variations of  $w$  affects variation of  $E$ ?*

# LMS with 1 non-linear unit

## Compute the gradient



Dip. Informatica  
University of Pisa

**Important Exercise:** compute the gradient of the error function

(do it before the next lecture)  $\frac{\partial E(\mathbf{w})}{\partial w_j}$  with  $o(\mathbf{x}_p) = \text{out}(\mathbf{x}_p) = f_\sigma(\mathbf{x}_p^T \mathbf{w}) = f_\sigma(\text{net}(\mathbf{x}_p))$

Result (1 unit, all the patterns  $p = 1..l$ ):

$$\frac{\partial E(\mathbf{w})}{\partial w_j} = -2 \sum_{p=1}^l (\mathbf{x}_p)_j (d_p - f_\sigma(\mathbf{x}_p^T \mathbf{w})) f'_\sigma(\mathbf{x}_p^T \mathbf{w}) = -2 \sum_{p=1}^l (\mathbf{x}_p)_j \boxed{\text{new } \delta_p} f'_\sigma(\text{net}(\mathbf{x}_p))$$

We can simplify the notation as:

$$\frac{\partial E(\mathbf{w})}{\partial w_j} = -2 \sum_{p=1}^l \boxed{\text{new } \delta_p} (d_p - o(\mathbf{x}_p)) f'_\sigma x_{p,j}$$

Note: here we have just 1 unit, more indices will be used with more units, see backprop.

# Gradient descent algorithm

- The same as for linear unit using the *new delta rule* (batch/on-line):

$$\mathbf{w}_{\text{new}} = \mathbf{w} + \eta \delta_p \mathbf{x}_p$$

↓

Pattern  $p$

The unique Unit has not  
index here

$$(d_p - f_\sigma(\text{net}(\mathbf{x}_p)))f'_\sigma(\text{net}(\mathbf{x}_p))$$

...or simplifying the notation

$$(d_p - o(\mathbf{x}_p))f'_\sigma$$

# Gradient descent algorithm as an error correction rule

- The same as for linear unit using the *new delta rule* (batch/**on-line**):

$$\mathbf{w}_{\text{new}} = \mathbf{w} + \eta \delta_p \mathbf{x}_p$$

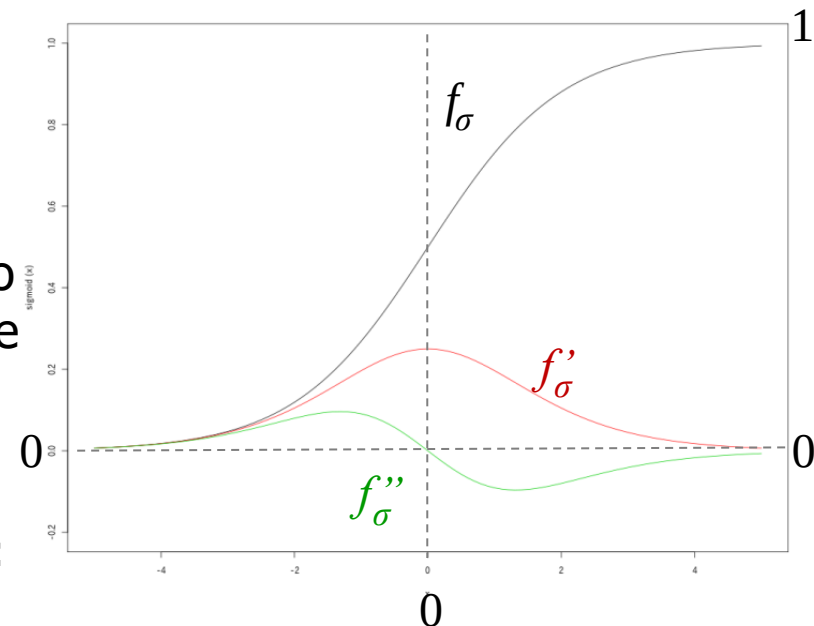
$$\delta_p = (d_p - o(x_p)) f'_\sigma$$

Again, an **error correction rule**. Moreover:

- The parameters  $a$  (slope of  $f$ ) can affect the step of gradient descent
- Max of  $f'_\sigma$  for inputs ( $net$ ) close to 0 ( $\sim$  linear unit). Here high value of the  $\delta$  are possible
- Minimum of  $f'_\sigma$  are for **saturated cases** (where  $f$  go to 0 or 1 asymptotically)  $\rightarrow$  better to avoid a premature saturation, small  $\delta$ , and hence very slow changes of  $w$ , at the beginning (starting with small weights) or later



- Insight: we see also a bridge between the two alg.s: toward no corrections for correct outputs also for LMS alg. (as it was for the perceptron learn. alg.). I.e. Using  $f_\sigma$  we are approximating not only the LTU but also the perceptron learn. alg.



Plot of  $f_\sigma$  (black)  $f'_\sigma$  (red)  $f''_\sigma$  (green)

# Neural Networks (NN)

All the ingredients are ready:

NN by two views of Multi-Layer Perceptron (MLP)-NN:

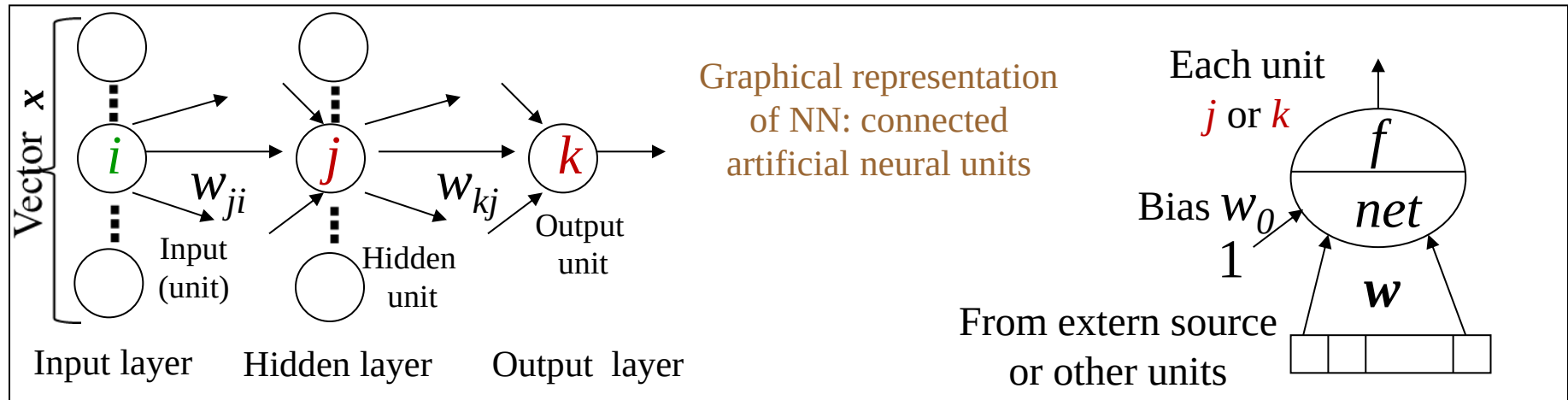
A. A **network** of interconnected units

B. A flexible **function**  $h(\mathbf{x})$  (as nested non-linear functions)

# The Neural Network (NN)

## Standard feedforward NN (with one hidden layer)

### A. A network of units



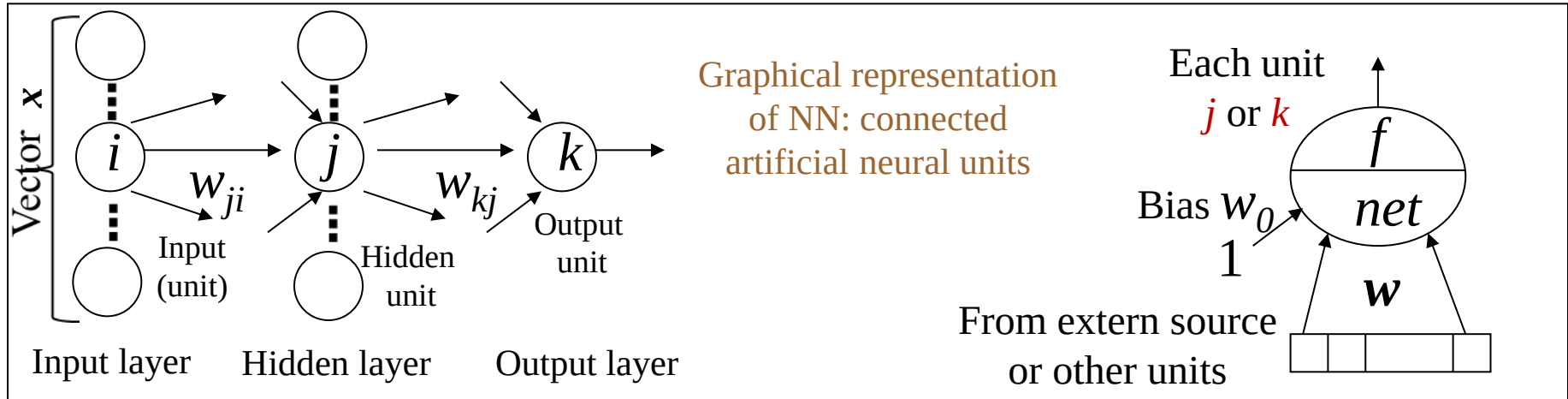
In a **MLP** (Multi Layer Perceptron) architecture

- the units are connected by **weighted links** and they are organized in the form of **layers**
- the **input layer** is simply the source of the input  $x$  that projects onto the hidden layer of units: it loads (copy) the (extern) input patterns  $x$  (the input units  $i$  do not compute the *net* and  $f$ )
- the **hidden layer** projects onto the **output layer** (feedforward computation of this two-layers network in the fig.) or to another hidden layer

# Neural Networks: **two views**

## Standard feedforward NN (with one hidden layer)

### A. A network of units



### B. A flexible function

$$h(\mathbf{x}) = f_k \left( \sum_j w_{kj} f_j \left( \sum_i w_{ji} x_i \right) \right)$$

Free parameters  
(weight matrix)

Non-linear function  
(sigmoid)

Input variables  
(vector  $\mathbf{x}$ )

Recall of a linear function

$$h(\mathbf{x}) = \sum_i w_i x_i$$

# Neural Networks: components

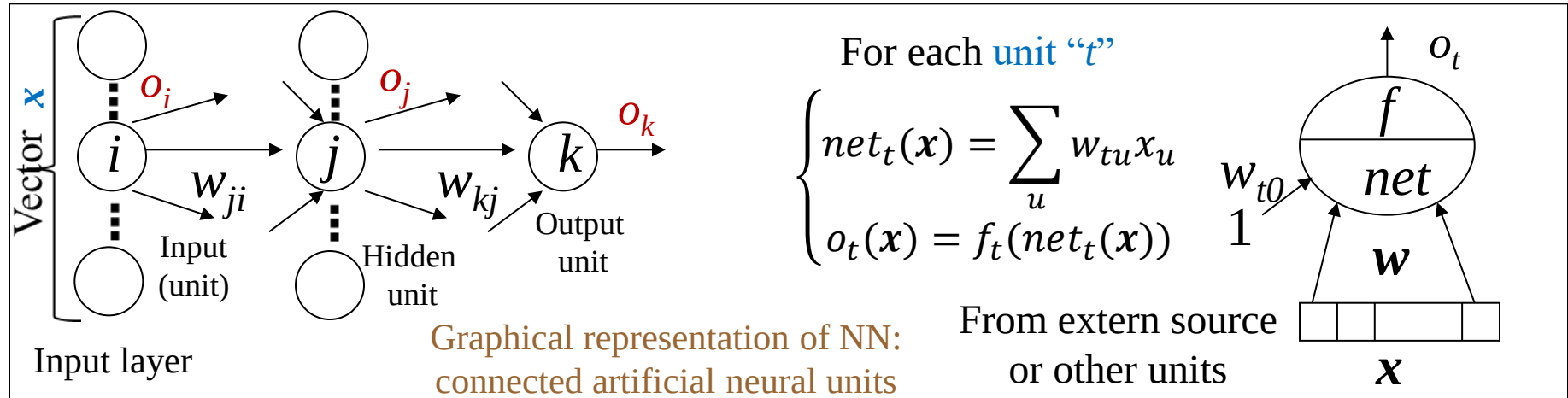
---

- NN model traditionally presented by the type of
  - **UNIT**: net, activation functions
  - **ARCHITECTURE**: number of units, topology (e.g. also number of layers)
  - **LEARNING ALGORITHM**

# The NN and their Units

## Standard feedforward NN (with one hidden layer)

### A. A network of units



Notation:

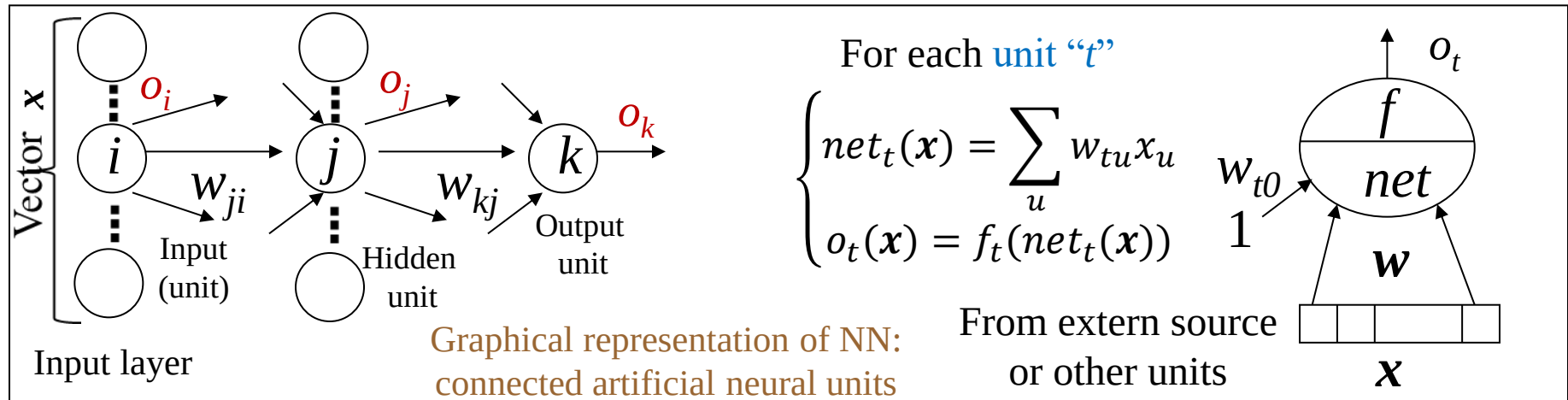
- The index  $t$  denotes a generic unit, it can be either  $j$  or  $k$  (\*)
- The index  $u$  denotes a generic input component, it can be either  $i$  or  $j$  (\*)
- In the unit,  $x$  is a generic input from an external source (input vector) or from other units (\*) *according to the position of the unit in the network*
- If we load the pattern  $x$  in the input layer, we can use the notation with  $o$  for both the inputs and the hidden units outputs. Hence, *inside the network*, the input to each unit  $t$  from *any source*  $u$  (through the connection  $w_{tu}$ ) is typically denoted as  $o_u$  (we will use this style in the back-prop derivation).



# The NN and their Units

## Exercise to use a uniform notation for the inputs

### A. A network of units



### ***Important Exercise*** (do it before the next lecture):

For the eq. of units “ $t$ ”, using the symbol  $o$  for any input to them:

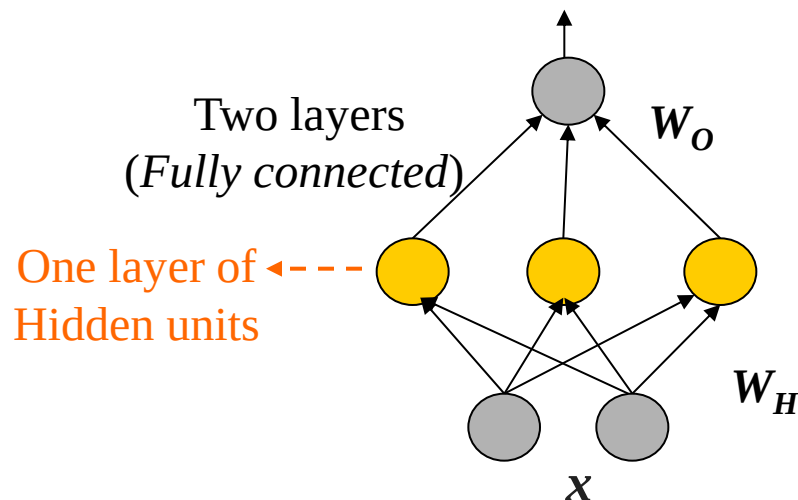
1. What is  $o_u$  for  $w_{ji}$ ?, i.e the input to the unit  $j$ ?
2. What is  $o_u$  for  $w_{kj}$ ?
3. What is  $w_{t0}$  for each unit  $t$ ?
4. Write the equations for a hidden unit “ $j$ ” ( $t=j$ ) and an output unit “ $k$ ” ( $t=k$ ), using the symbol  $o$  for the any *input* to them (either extern source or other units).



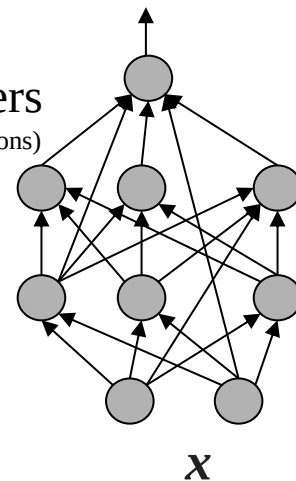
# MLP Architecture

## Standard feedforward NN

- The *architecture* of a NN defines the topology of the connections among the units.
- The two-layer **feedforward neural network** described in Equation  $h(x)$  corresponds to the well-know **MLP** (Multi Layer Perceptron) architecture:



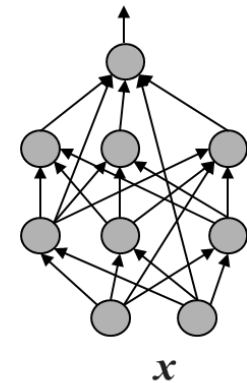
Three Layers  
(with other connections)



# Feedforward processing

The processing of a pattern for feedforward NN precedes as in the following (from the input layer to the output layer):

- For each input pattern  $x$ 
  1. the input pattern is load in the input layer
  2. we compute the output of all the units of the 1st hidden layer
  3. we compute the output of all the units of the 2st hidden layer and so on for all the hidden layers
  4. we compute the output of all the units of the output layer (NN output  $h(x)$ )
  5. we can now compute the error (delta) at the output level



# Architectures:

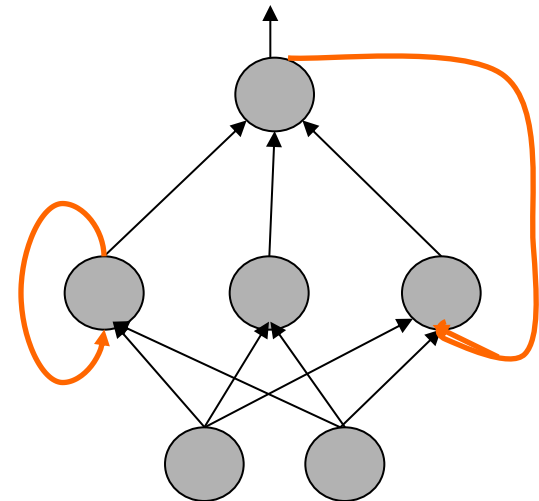
## Feedforward versus Recurrent



- **Feedforward:** direction: input  $\rightarrow$  output
- **Recurrent** neural networks: A different category of architecture, based on the addition of *feedback loops* connections in the network topology,
  - The presence of **self-loop** connections provides the network with dynamical properties, letting a memory of the past computations in the model.
  - This allows us to extend the representation capability of the model to the processing of sequences (and structured data).

### Recurrent neural networks:

- See separate slides in this course
- They will be the subject (further developed) of the CNS (Computation Neuroscience) course.



# Analysis and questions

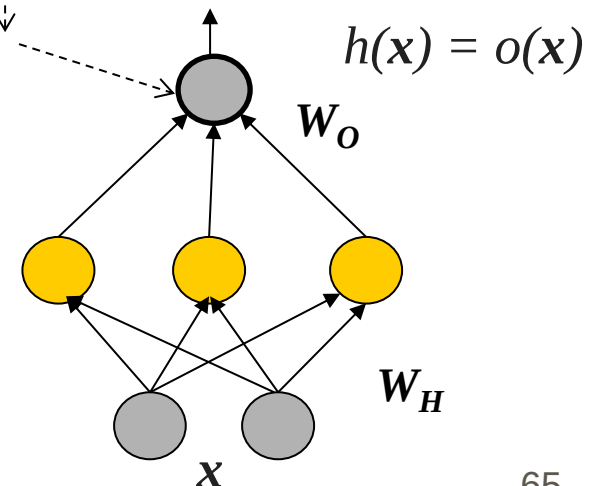
---

- 1. Why NN is a flexible model? (why does it work well)?**
  - Which tasks?
  - Can be interpreted as **LBE**? (an important connection for us)
- 2. Is the flexibility theoretically grounded?**
- 3. How to learn the weights of a NN?**

# 1. Notes on NN flexibility: Which Tasks?

- **Hypothesis space:** *continuous* space of all the functions that can be represented by assigning the weight values of the given architecture.
- Note that, depending on the class of values produced by the network output units, discrete or continuous, the model can deal, respectively, with **classification tasks** (sigmoidal output *f.*) or **regression tasks** (linear output *f.*) tasks.

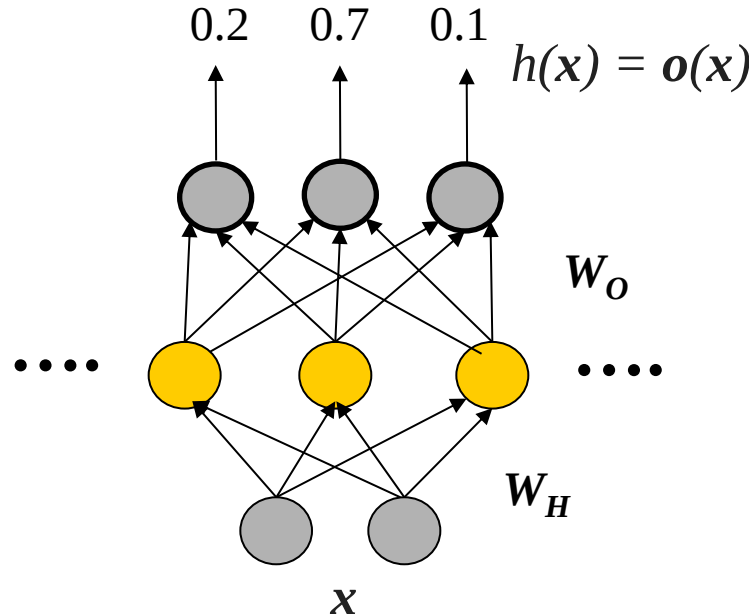
$$h(\mathbf{x}) = \underbrace{f_k}_{\text{output function}} \left( \sum_j w_{kj} f_j \left( \sum_i w_{ji} x_i \right) \right)$$



# 1. Notes on NN flexibility

## Which Tasks?

- Also **multi-regression** or **multi-classes classifier** can be obtained by using **multiple output units**



# 1. NN as a *function*

$$h(\mathbf{x}) = f_k \left( \sum_j w_{kj} f_j \left( \sum_i w_{ji} x_i \right) \right)$$

- This is the *function* computed by a two-layer feedforward neural network
- Units and architecture just a graphical representation (of the data flow process)
- Each  $f_j \left( \sum_i w_{ji} x_i \right)$
- can be seen as computed by an independent processing element (unit, a hidden unit), or a **special kind of phi** ( $\phi$ ) of LBE
- Also, NN is a function **non linear** in the parameters  $\mathbf{w}$

# 1. NN as a dictionary approach (LBE) (1, equations)

- **LBE:** (Recall) **fixed** linear basis functions

$$h(\mathbf{x}) = \sum_j w_j \phi_j(\mathbf{x})$$

Regression

$$h(\mathbf{x}) = f\left(\sum_j w_j \phi_j(\mathbf{x})\right)$$

Classification

(either regression with  $f$  = identity function)

- **Neural Network**

$$h(\mathbf{x}) = f_k\left(\sum_j w_{kj} f_j\left(\sum_i w_{ji} x_i\right)\right) = f\left(\sum_j w_j \phi_j(\mathbf{x}, \mathbf{w})\right)$$

using

$$\phi_j(\mathbf{x}, \underline{\mathbf{w}}) = f_j\left(\sum_i w_{ji} x_i\right)$$

hidden units' output

because now  $\phi$  depends also on  $\mathbf{w}$

# 1. NN

$$h(\mathbf{x}) = f_k \left( \sum_j w_{kj} f_j \left( \sum_i w_{ji} x_i \right) \right)$$

## as a dictionary approach (LBE) (2, discussion)

- **LBE:** (Recall) **fixed** linear basis functions (phi preselected: does not change with training data),

— And linear w.r.t. parameters  $w$ :

$$\sum_j w_j \underbrace{\phi_j(\mathbf{x})}$$

- **NN: Adaptive (flexible) basis functions approach:**

the basis functions themselves are **adapted** to data (by fitting the  $w$  in phi)

- Note that we fix the same type of basis functions for all the terms in the basis expansion (given by the activation function)

$$\phi_j(\mathbf{x}, \underline{\mathbf{w}}) = f_j \left( \sum_i w_{ji} x_i \right)$$

- $h(\mathbf{x})$  as (nonlinear function of weighted) sums of nonlinearly transformed linear models + the important enhancement of adaptivity



Please come back to these slides after SVM (another form of LBE) to compare

# 1. Hidden Layer Relevance and Interpretation



Dip. Informatica  
University of Pisa

- Each basis function (**hidden unit**) computes a **new** nonlinear derived features, **adaptively** (by learning, according to the training data)
- I.e. the parameters of the basis function  $w$  are learned from data by learning

$$\phi_j(\mathbf{x}, \underline{\mathbf{w}}) = f_j \left( \sum_i w_{ji} x_i \right)$$

In other words:

- The representational capacity of the model is related to the presence of a hidden layer of units, with the use of non-linear activation function, that transforms the input pattern into the **internal representation** of the network.
- The learning process can define a suitable internal representation, also visible as new hidden features of data, allowing the model to extract from data the higher-order statistic that are relevant to approximate the target function.

# 1. Note: Non-linear hidden layer is need



Dip. Informatica  
University of Pisa

- Non-linear units are essential:  
MLP with linear units = 1 layer NN !

$$h(\mathbf{x}) = f_k \left( \sum_j w_{kj} f_j \left( \sum_i w_{ji} x_i \right) \right)$$

- Exercise: using  $f$  as identity function you can rewrite  $h(\mathbf{x})$  using a singular layer of units
- Anyway for the learning algorithm this introduces an issue: The model is non-linear in the parameters, i.e. w.r.t. to  $w \rightarrow$  we have a non linear optimization problem

# Analysis and questions

---

**1. Why NN is a flexible model? (why does it work well)?**

- Can be interpreted as LBE?

**2. Is the flexibility theoretically grounded?**

**3. How to learn the weights of a NN?**

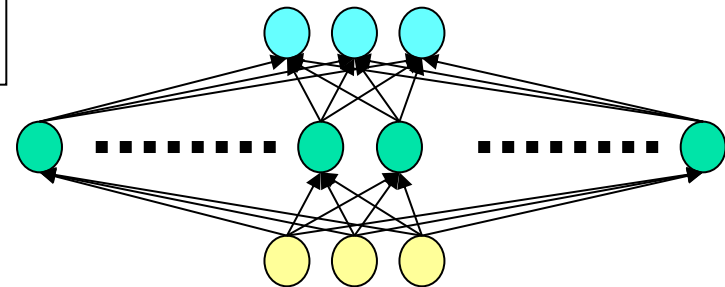
## 2. Universal approximation (important!)

- Many early results (Cybenko 1989, Hornik et al. 1993, etc...)
- Shortly:
  - A single hidden-layer network (with logistic activation functions) can approximate (arbitrarily well) every continuous (on hyper cubes) function (provided enough units in the hidden layer)  
[e.g. image it generalizes approximation by finite Fourier series]
  - A MLP network can approximate (arbitrarily well) every input-output mapping (provided enough units in the hidden layers)

Given  $\varepsilon$ ,  $\exists h(\mathbf{x})$  s.t.  $|(f(\mathbf{x})-h(\mathbf{x}))| < \varepsilon$   
 $\forall \mathbf{x}$  in the hypercube

$$h(\mathbf{x}) = f_k \left( \sum_j w_{kj} f_j \left( \sum_i w_{ji} x_i \right) \right)$$

linear



*Existence theorem!* This not provide the algorithm nor the number of units



# Two fundamental issues

---

After this fundamental result (MLP is able to *represent* any function), two issues will deserve our attention:

- How to learn by NN → next lecture for the first alg.
- How to decide a NN architecture → next lectures

# NN Expressive power

- The ***expressive power*** of NN is strongly influenced by two aspects: the number of units and their configuration (architecture)
- The number of units can be related to the discussion of the **VC-dimension** of the model.  
Specifically, the network capabilities are influenced by the number of parameters  $w$ , that is proportional to the number of units \*, and further studies report also the dependencies on their value sizes, e.g.:
  - Weights = 0  $\rightarrow$  minimal VC-dim
  - Small weights  $\rightarrow$  linear part of the activation function (small VC-dim)
  - Higher weights values  $\rightarrow$  more complex model

\* Number of parameters (fully-conn. MLP) = #input-units X #hidden-units X #output-units

- Next lectures: NN as a *variable-size hypothesis space*
- Indeed, we start with weight values near 0, then  $w$  values increase during training  $\rightarrow$  higher VC-dim  $\rightarrow$  it is a variable size hp space



# How many layers?

A look ahead (toward **deep learning**).

- The univ. approx. theorem is a fundamental contribution
- It shows that 1 hidden layer is sufficient in general, but it does **not** assure that a “small number” of units could be sufficient (it does not provide a limit on such number)
- It is possible to find boundaries on such number (for many f. families)
- But also to find “*no flattening*” results (on efficiency, not expressivity):
  - cases for which the implementation by a single hidden layer would require an exponential number of units (w.r.t *input dim.*), or non-zero weights,
  - while more layers can help (it can help for the number of units/weights and/or for learning such approximation)
  - But is it easy to optimize (training) a MLP with many layers?
- See later for deep learning.

# On the inductive bias of NN

- NN with backpropagation learning algorithm:
- Generally related to the ***smoothness*** properties of functions:
  - Small input variations  $\rightarrow$  small output variations
  - E.g. a locally limited value of the first derivative
  - A very common assumption in ML
- Why make sense? A target function that is **Non-smooth**: random number generator  $\rightarrow$  generalization cannot be achieved !

# Analysis and questions

---

- 1. Why NN is a flexible model? (why does it work well)?**
  - Can be interpreted as LBE?
- 2. Is the flexibility theoretically grounded?**
- 3. How to learn the weights of a NN?**

# 3. NN Learning Algorithm



Dip. Informatica  
University of Pisa

- Repetita: **The learning algorithm** allows adapting the free-parameters  **$w$**  of the model, i.e. the values of the connection-weights, in order to obtain the best approximation of the target function.
- As usual, we will realize this in terms of minimization of an error (or loss) function on the training data set.

# From Perceptron to NN

- **Perceptron:**

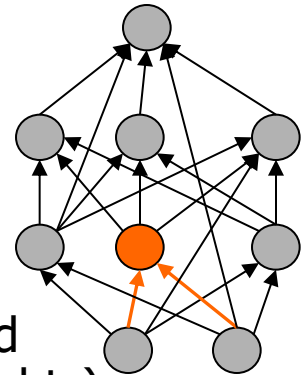
- We have a Perceptron learning algorithm (delta rule)
- cannot represent all the Boolean functions (e.g. XOR)

- A **network of Perceptrons** can represent every Boolean function
- Problem: define a **Learning Algorithm for the network (MLP)**

- *What is different? Credit assignment problem:*

- Which credit to the **hidden units**?
- not easy (as for single Perceptron) when error signal is not directly measurable: we *don't* know the error(**delta**)/desired response for the **hidden units** (useful to change their weights).

- (Repetita) Non-linear wrt to  $w \rightarrow$  *non linear optimization problem*
- Supposed too difficult by Minsky-Papert (1969), while faced by many researchers (see historical notes) with the *backpropagation* algorithm popularized by Rumelhart, Hinton, Williams in the PDP book (1986)  
 $\rightarrow$  renaissance of NN



# The loading problem

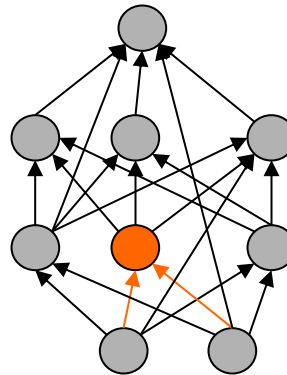
Note that in general, it is indeed a difficult problem:

- *Loading problem*: ("loading" a given set of TR data into the free par. of the NN)
  - given a network and a set of examples
  - answer yes/no: is there a set of weights so that the network will be consistent with the examples?
- The loading problem is *NP-complete* (Judd, 1990), it is *not* known a polynomial alg. to solve it.
- In practice networks can be trained in a reasonable amount of time (see the back prop alg.) although optimal solution is not guaranteed.

# How to solve?

## Key steps:

- Credit assignment problem: how to change the hidden layer weights?
- The ***gradient descend approach***, minimizing a loss function, can be extended to MLP, provided that the *loss function* and *the activation functions* are *differentiable functions*
  - So, to find the **delta** for every units in the network



# Intro to Backpropagation (BP)

- **The problem is the same we have seen** for the other models, starting again with the **LMS** approach (error/data term):
  - **Given** a set of  $l$  training example  $(\mathbf{x}_p, d_p)$  and a (inner) loss measure  $L$  (e.g.  $L(h(\mathbf{x}_p), d_p) = (d_p - h_{\mathbf{w}}(\mathbf{x}_p))^2$  for the MSE)
  - **Find:** The weight vector  $\mathbf{w}$  that minimizes the expected error on the training data (we first focus on the data term), **by computing the gradient of the error function :**

$$E(\mathbf{w}) = R_{emp} = \frac{1}{l} \sum_{p=1}^l (d_p - h(\mathbf{x}_p))^2$$

- **What we need:** differentiable loss, differentiable activation functions, a network to follow the information flow (!)

- **Nice properties of the backprog. alg.** (also for programming):
  - Easy because of the *compositional* form of the model
  - It keeps track only of quantities local to each unit (by local variables) → modularity of the units is preserved
  - *Efficiency*:  $O(\#W)$  instead of  $O(\#W^2)$  (factorization of deltas, see later)
  - On the *biological plausibility*: controversial, but it has been supposed that in the brain, to learn, we have a local suboptimal approx. of BP

# Backproagation documents (**next lecture**)



Dip. Informatica  
University of Pisa

- **GO TO PDF file: \*Backprop\*.pdf**

We will develop it in the *next class*

See (as basic bibliography):

- Course Latex notes (\*Backprop\*notes...latex\*.pdf")
- D. E. Rumelhart, J. L. McClelland. Parallel Distributed Processing, Vol.1, by, MIT Press: Chapter 8 by Rumelhart, Hinton, Williams (1986): "*Learning internal representations by error propagation*", pages: 322-328
  - You can find a pdf on the web, searching for the pdf of "Learning internal representations by error propagation"
  - E.g. a document with exactly the same content (but the links change continuously): [https://web.stanford.edu/class/psych209a/ReadingsByDate/02\\_06/PDPVolIChapter8.pdf](https://web.stanford.edu/class/psych209a/ReadingsByDate/02_06/PDPVolIChapter8.pdf)
- Prepare yourself with the **exercises** suggested in this class!
  - 1 volunteer for in-class solution via Teams (with an electr. board): a plus!

# Issues in Training NN



- Heuristic guidelines in setting the back-prop training

General problems:

- The model is often overparametrized
- Optimization problem is not convex and potentially unstable
- We discuss few of them (see e.g. Haykin for details)
  - Hyperparameters: Starting values, learning rate, on-line/batch
  - Overfitting and regularization
  - Input scaling/ Output representation
  - Hyperparameters: Number of hidden units
  - Multiple Minima
  - Stopping criteria
  - .....

**See next lectures** (after the backprogratation lecture)

# Q&A lecture set-up

- Please send to me your questions by **email** so to set up Q&A lectures

Please, **USE in the subject the TAG [ML-24-Q]**

- We can discuss
  - Self-evaluation quiz results
  - New questions to better understand some parts of the course (no questions, no Q&A lecture ;-) )
- Memento
  - Special interactive classes will be used to assess activities and to make a ML discussion forum (**Q&A classes**).  
This makes the course a class, not just a set of records
  - Within a **cooperative** and collaborative context!

# Bibliography (NN: first part)

- Haykin (2<sup>nd</sup> ed.):
  - chapters 1, 3 (with convergence theorem)
  - chapter 4 (with MLP/backprop alg.)
- Haykin (3<sup>rd</sup> ed.):
  - chapters Introduction, 1 (with convergence theorem)
  - chapter 4 (with MLP/backprop alg.)
- Mitchell: chapter 4

Other: Every rigorous NN/ML book ! E.g.

- Hastie, Tibshirani, Friedman: cap 11
- Bishop
- .....

For the next lecture:

- Parallel Distributed Processing, Vol.1, by D. E. Rumelhart, J. L. McClelland, MIT Press (an historical book): **chap. 8** (see link to pdf in previous slides)